Computer Vision with

Open CV LAB Experiments

1. Perform basic Image Handling and processing operations on the image. a. Read an image in python and Convert the given Image into Grayscale

b. Read an image in python and Convert an Image to Blur using GaussianBlur.

c. Read an image in python and Convert the given Image to show outline using Canny function.

d. Read an image in python and Dilate an Image using Dilate function.

e. Read an image in python and Erode an Image using erode function.

2. Perform basic video processing operations on the captured video

• Read captured video in python and display the video, in slow motion and in fast motion. 3. Capture video from web Camera and Display the video, in slow motion and in fast motion. 4. Scaling an image to its Bigger and Smaller sizes.

5. Perform Rotation of an image to clockwise and counter clockwise direction. 6. Perform moving of an image from one place to another.

7. Perform Affine Transformation on the image.

8. Perform Perspective Transformation on the image.

9. Perform Perspective Transformation on the Video.

10. Perform transformation using Homography matrix.

11. Perform transformation using Direct Linear Transformation.

12. Perform Edge detection using canny method

13. Perform Edge detection using Sobel Matrix along X axis

14. Perform Edge detection using Sobel Matrix along Y axis

15. Perform Edge detection using Sobel Matrix along XY axis

16. Perform Sharpening of Image using Laplacian mask with negative center coefficient.

0	1	0
1	−4	1
0	1	0

17. Perform Sharpening of Image using Laplacian mask implemented with an extension of diagonal neighbors,

1	1	1
1	−8	1
1	1	1

18. Perform Sharpening of Image using Laplacian mask with positive center

Mask of Laplacian + addition

$$
\begin{aligned}
g(x,y) &= f(x,y) - [f(x+1,y) + f(x-1,y) \\
&\quad + f(x,y+1) + f(x,y-1) + 4f(x,y)] \\
&= 5f(x,y) - [f(x+1,y) + f(x-1,y) \\
&\quad + f(x,y+1) + f(x,y-1)]
\end{aligned}
$$

0	-1	0
-1	5	-1
0	-1	0

coefficient.

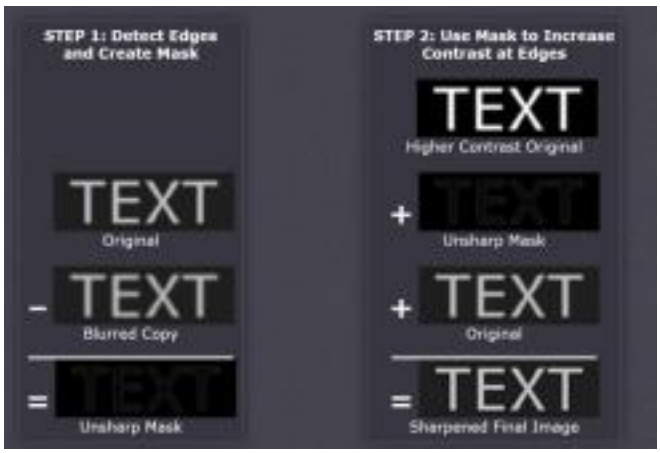
19. Perform Sharpening of Image using unsharp masking.

Unsharp masking

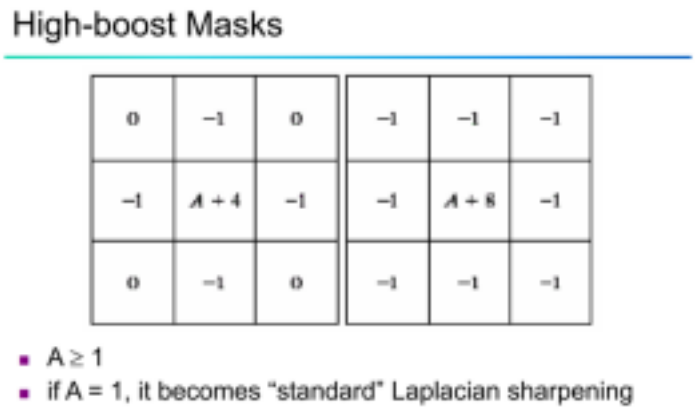
$$f_s(x,y) = f(x,y) - \bar{f}(x,y)$$

sharpened image = original image – blurred image

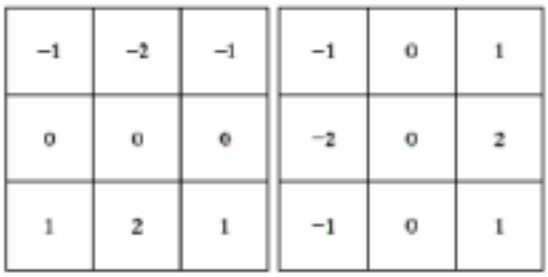
- to subtract a blurred version of an image produces sharpening output image.



20. Perform Sharpening of Image using High-Boost Masks.



21. Perform Sharpening of Image using Gradient masking.



22. Insert water marking to the image using OpenCV.

22. Do Cropping, Copying and pasting image inside another image using OpenCV. 23. Find the boundary of the image using Convolution kernel for the given image. 24. Morphological operations based on OpenCV using Erosion technique. 25. Morphological operations based on OpenCV using Dilation technique. 26. Morphological operations based on OpenCV using Opening technique. 27. Morphological operations based on OpenCV using Closing technique. 28. Morphological operations based on OpenCV using Morphological Gradient technique.

29. Morphological operations based on OpenCV using Top hat technique. 30.

Morphological operations based on OpenCV using Black hat technique. 31. Recognise watch from the given image by general Object recognition using OpenCV.

32. Using Opencv play Video in Reverse mode.

33. Face Detection using Opencv.

34. Vehicle Detection in a Video frame using OpenCV .

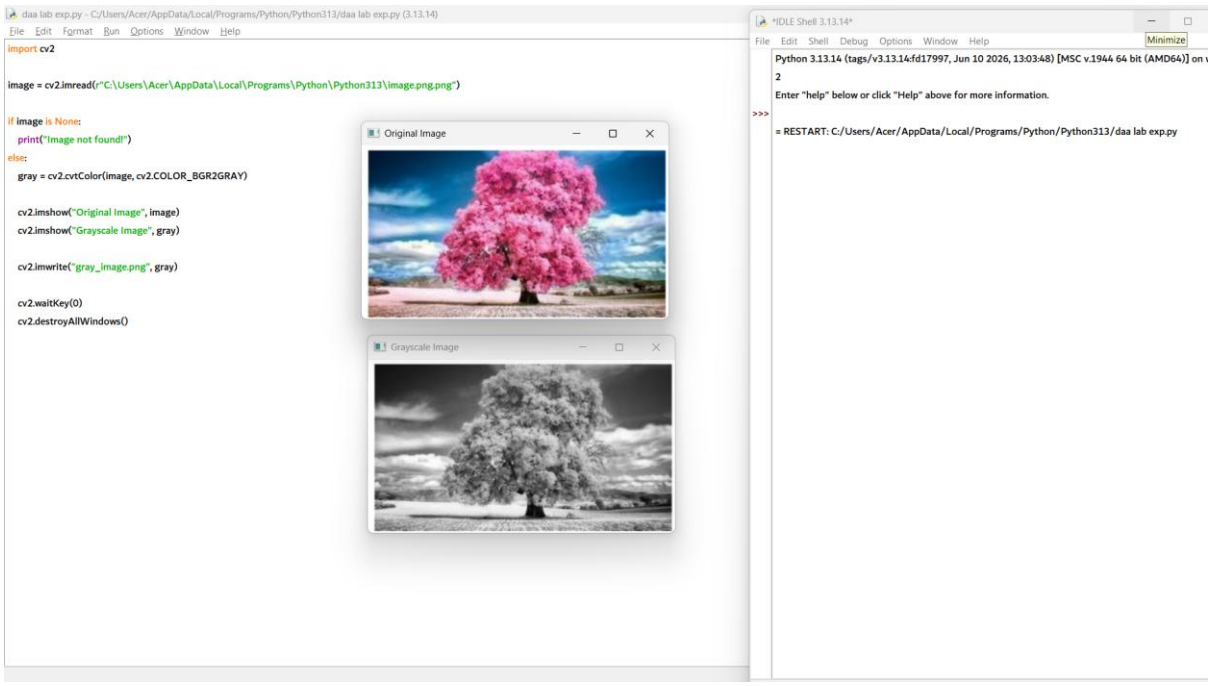
35. Draw Rectangular shape and extract objects.

1)

a. import cv2

```
image =
cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")


if image is None:
    print("Image not found!")
else:
```

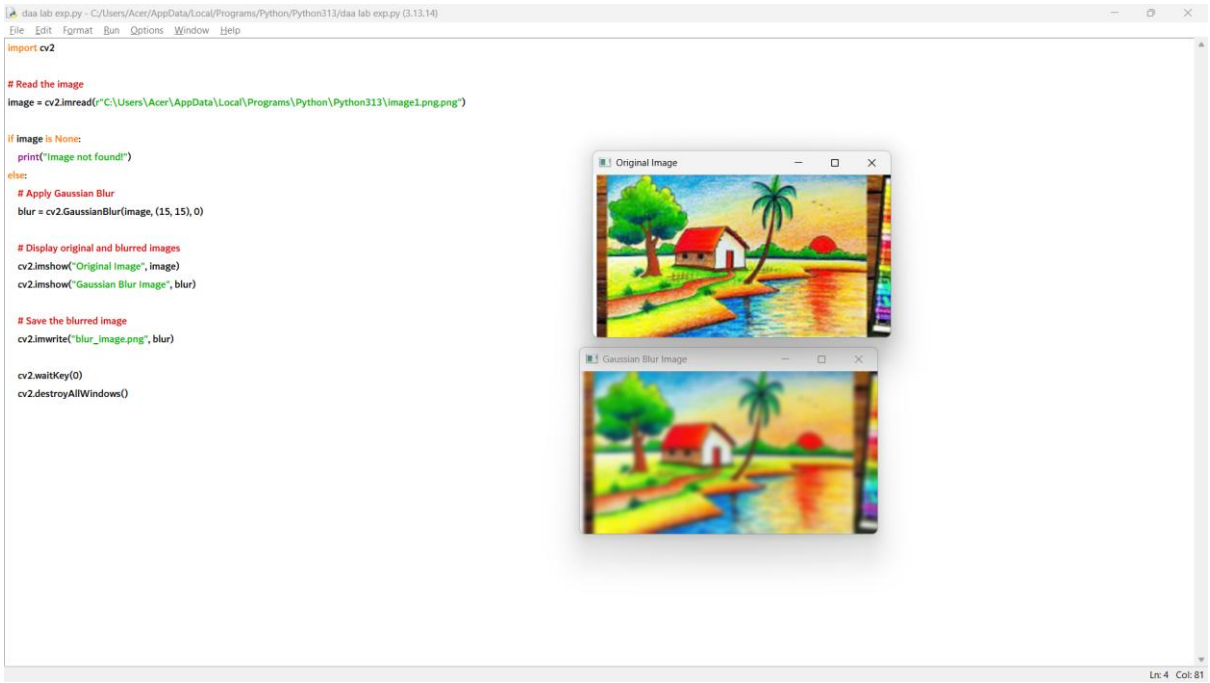
```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

cv2.imshow("Original Image", image)
cv2.imshow("Grayscale Image", gray)

cv2.imwrite("gray_image.png", gray)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



b. import cv2

```
# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image1.png.png")

if image is None:
    print("Image not found!")
else:
```

```
# Apply Gaussian Blur
blur = cv2.GaussianBlur(image, (15, 15), 0)

# Display original and blurred images
cv2.imshow("Original Image", image)
cv2.imshow("Gaussian Blur Image", blur)

# Save the blurred image
cv2.imwrite("blur_image.png", blur)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

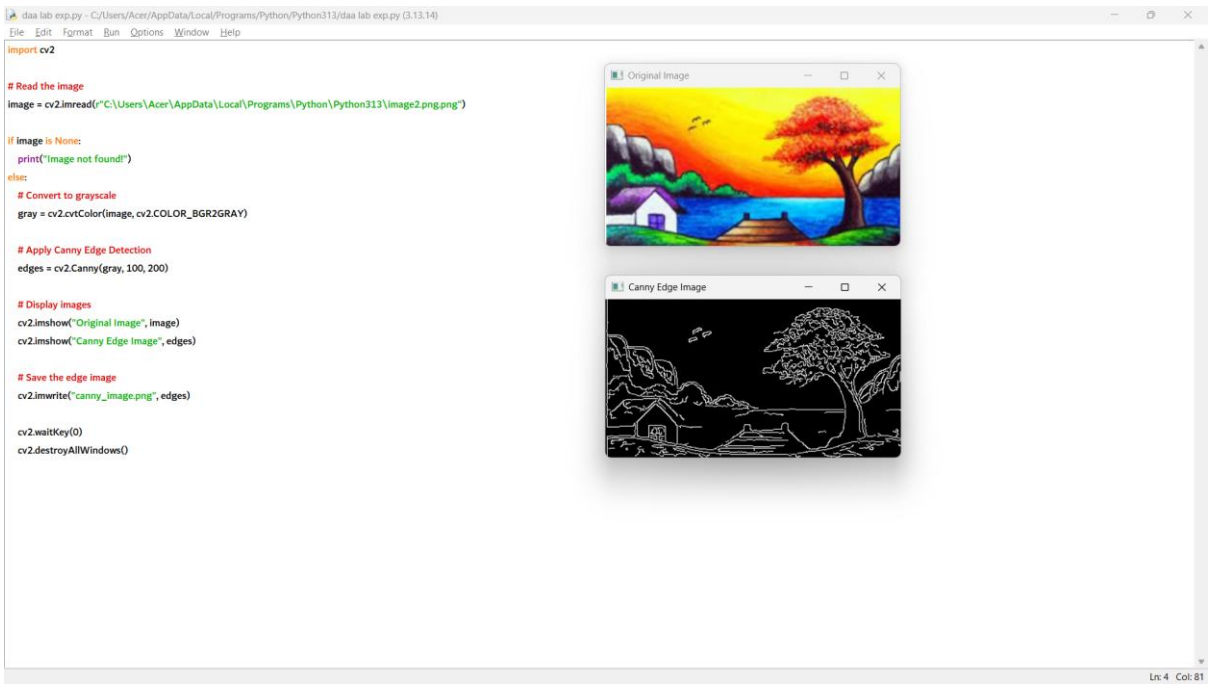


c. import cv2

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

```
if image is None:
    print("Image not found!")
else:
    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Apply Canny Edge Detection
    edges = cv2.Canny(gray, 100, 200)

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Canny Edge Image", edges)

    # Save the edge image
    cv2.imwrite("canny_image.png", edges)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



d. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Create a 5x5 kernel

    kernel = np.ones((5,5), np.uint8)

    # Dilate the image

    dilated = cv2.dilate(image, kernel, iterations=1)

    # Display images

    cv2.imshow("Original Image", image)

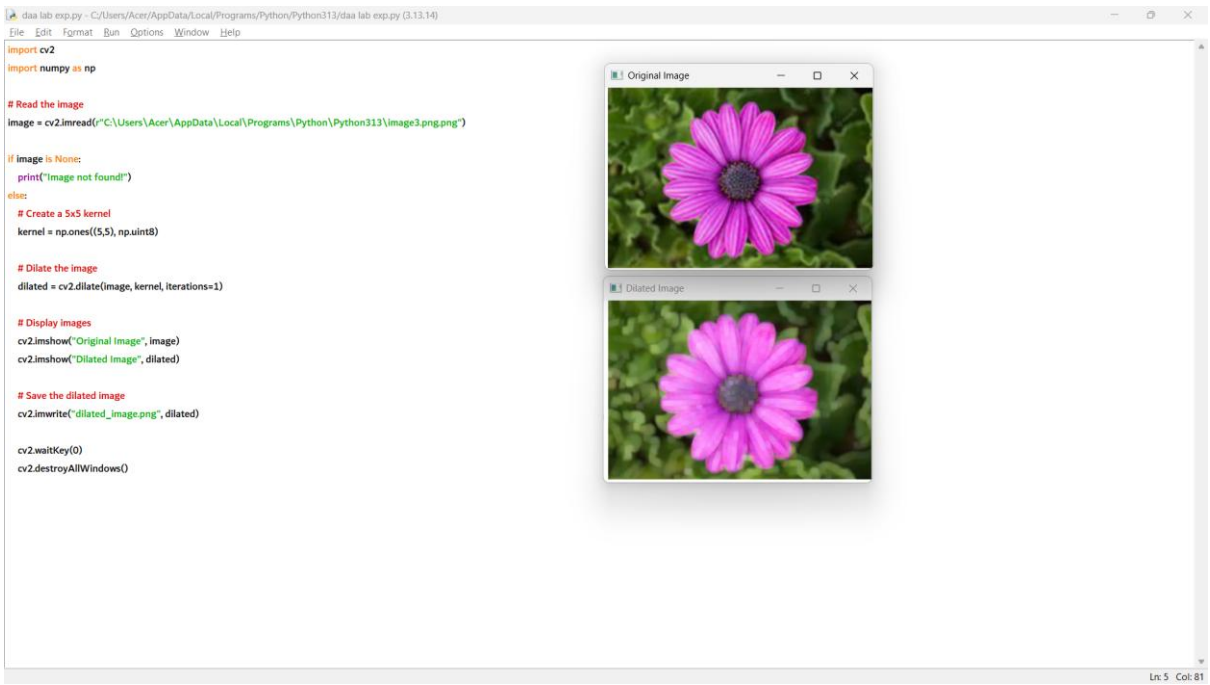
    cv2.imshow("Dilated Image", dilated)

    # Save the dilated image

    cv2.imwrite("dilated_image.png", dilated)

    cv2.waitKey(0)

    cv2.destroyAllWindows()



e. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Create a 5x5 kernel

    kernel = np.ones((5,5), np.uint8)

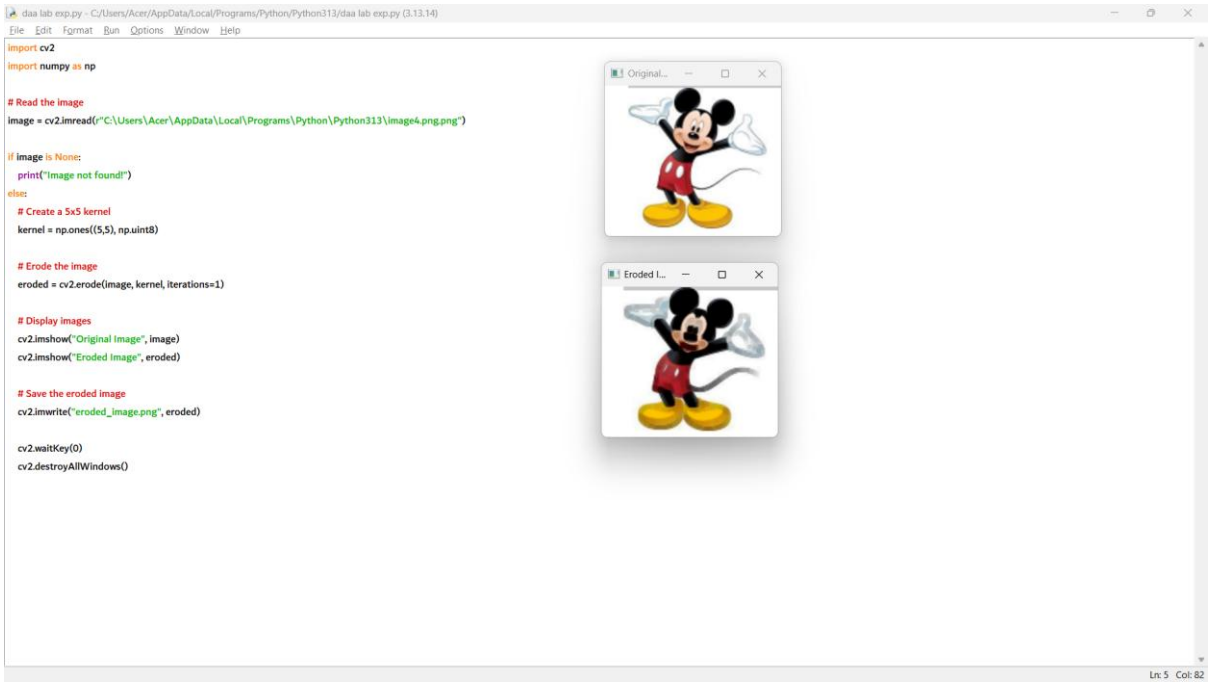
    # Erode the image

    eroded = cv2.erode(image, kernel, iterations=1)

    # Display images

    cv2.imshow("Original Image", image)

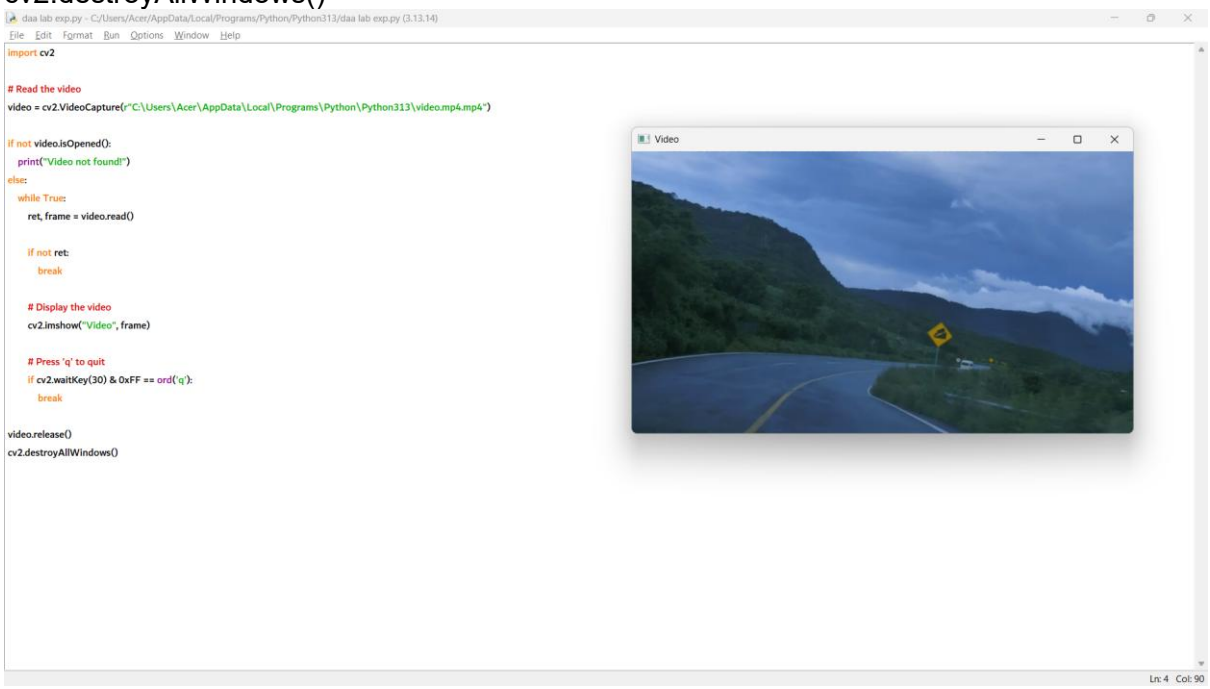
```
cv2.imshow("Eroded Image", eroded)

# Save the eroded image
cv2.imwrite("eroded_image.png", eroded)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



2. import cv2

```
# Read the video
video = cv2.VideoCapture(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\video.mp4.mp4")

if not video.isOpened():
    print("Video not found!")
else:
```

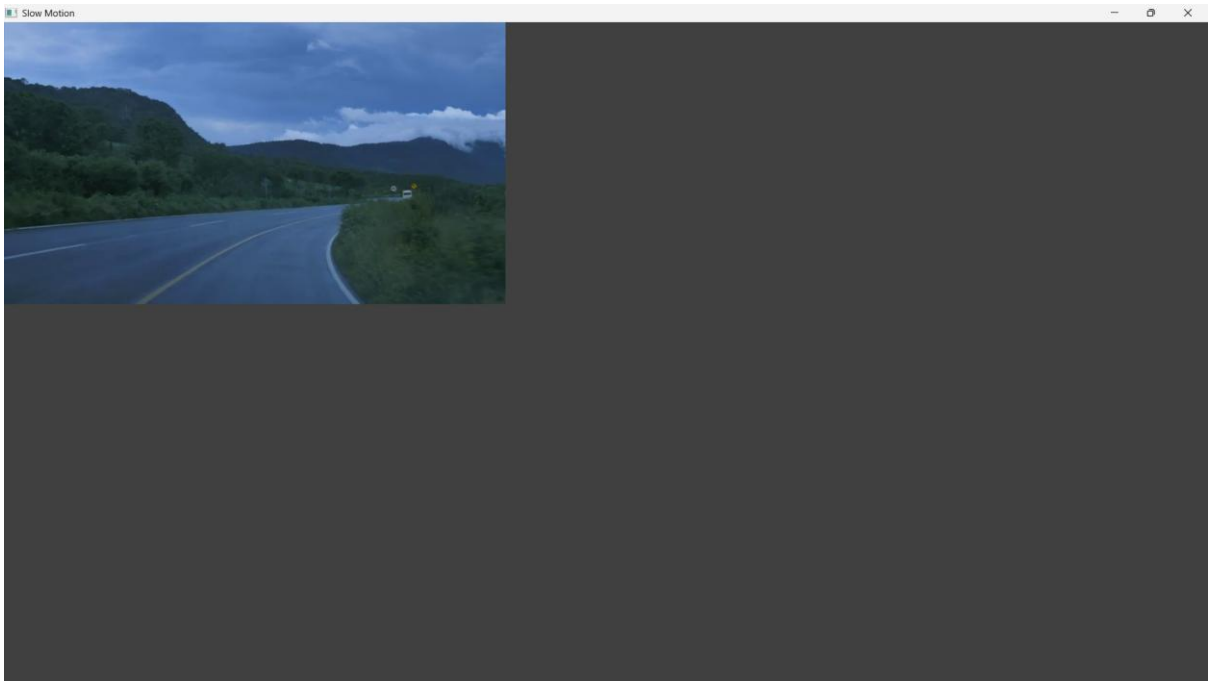
```
    while True:
        ret, frame = video.read()

        if not ret:
            break

        # Display the video
        cv2.imshow("Video", frame)

        # Press 'q' to quit
        if cv2.waitKey(30) & 0xFF == ord('q'):
            break

video.release()
cv2.destroyAllWindows()
```



Slow Motion Program

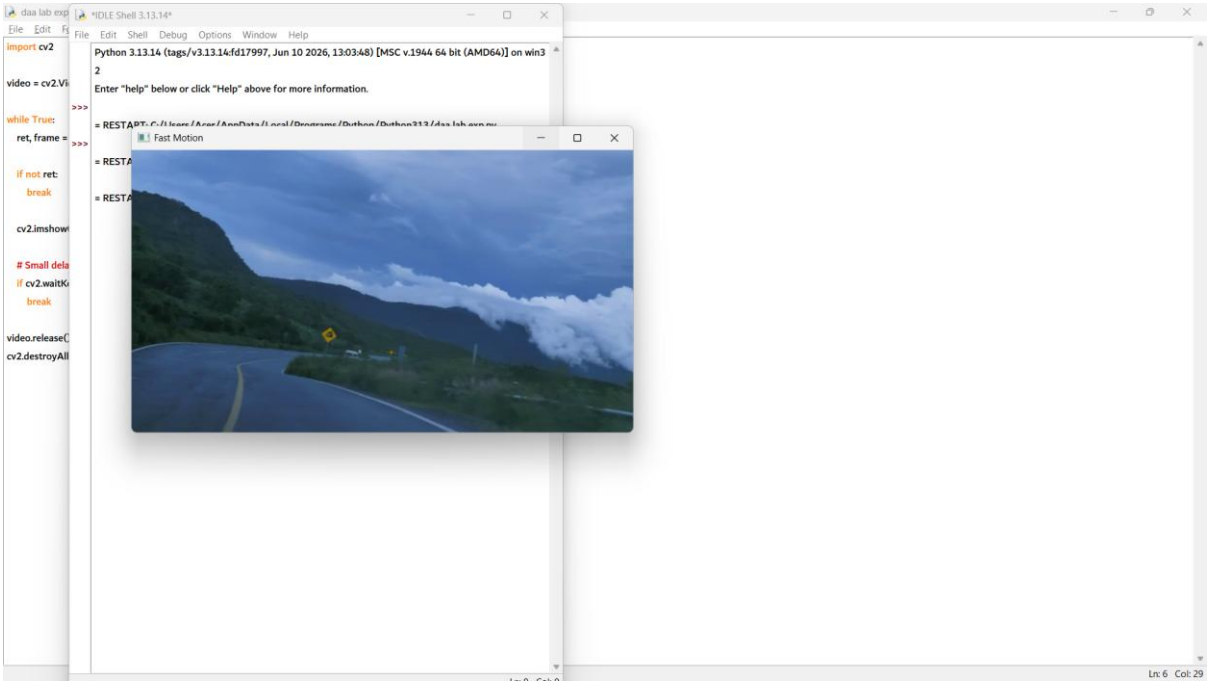
```
import cv2
```

```
video =
cv2.VideoCapture(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\video.mp4"
)


while True:
    ret, frame = video.read()


    if not ret:
        break


    cv2.imshow("Slow Motion", frame)


    # Delay increased for slow motion
    if cv2.waitKey(100) & 0xFF == ord('q'):
        break


video.release()
cv2.destroyAllWindows()
```



Fast Motion Program

```
import cv2


video =
cv2.VideoCapture(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\video.mp4"
)


while True:
    ret, frame = video.read()


    if not ret:
        break


    cv2.imshow("Fast Motion", frame)


    # Small delay for fast motion
    if cv2.waitKey(5) & 0xFF == ord('q'):
        break


video.release()
cv2.destroyAllWindows()
```



4.

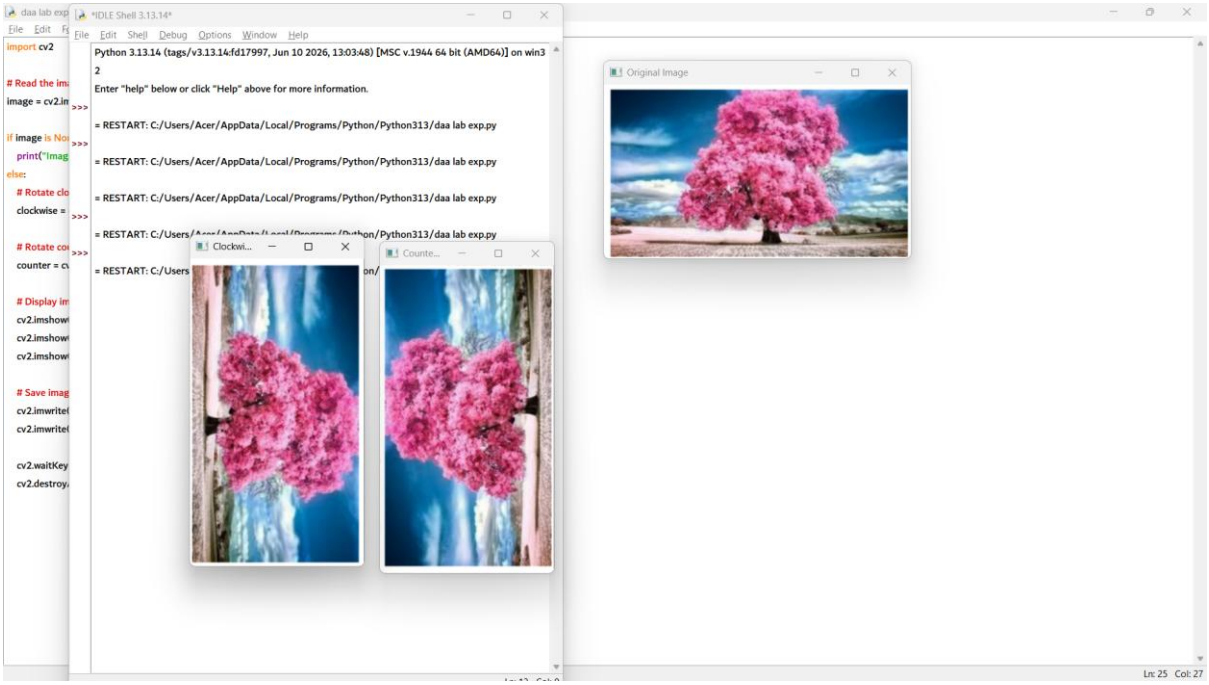
```
import cv2

# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:
    print("Image not found!")
else:
    # Scale to bigger size (2 times)
    bigger = cv2.resize(image, None, fx=2, fy=2)

    # Scale to smaller size (Half)
    smaller = cv2.resize(image, None, fx=0.5, fy=0.5)
```

```
# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Bigger Image", bigger)
cv2.imshow("Smaller Image", smaller)

# Save images
cv2.imwrite("bigger_image.png", bigger)
cv2.imwrite("smaller_image.png", smaller)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

5. import cv2

```
# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")
```

```
if image is None:
    print("Image not found!")
else:
    # Rotate clockwise (90°)
    clockwise = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)

    # Rotate counter-clockwise (90°)
    counter = cv2.rotate(image, cv2.ROTATE_90_COUNTERCLOCKWISE)

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Clockwise Rotation", clockwise)
    cv2.imshow("Counter Clockwise Rotation", counter)

    # Save images
    cv2.imwrite("clockwise.png", clockwise)
    cv2.imwrite("counterclockwise.png", counter)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



6. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Translation matrix

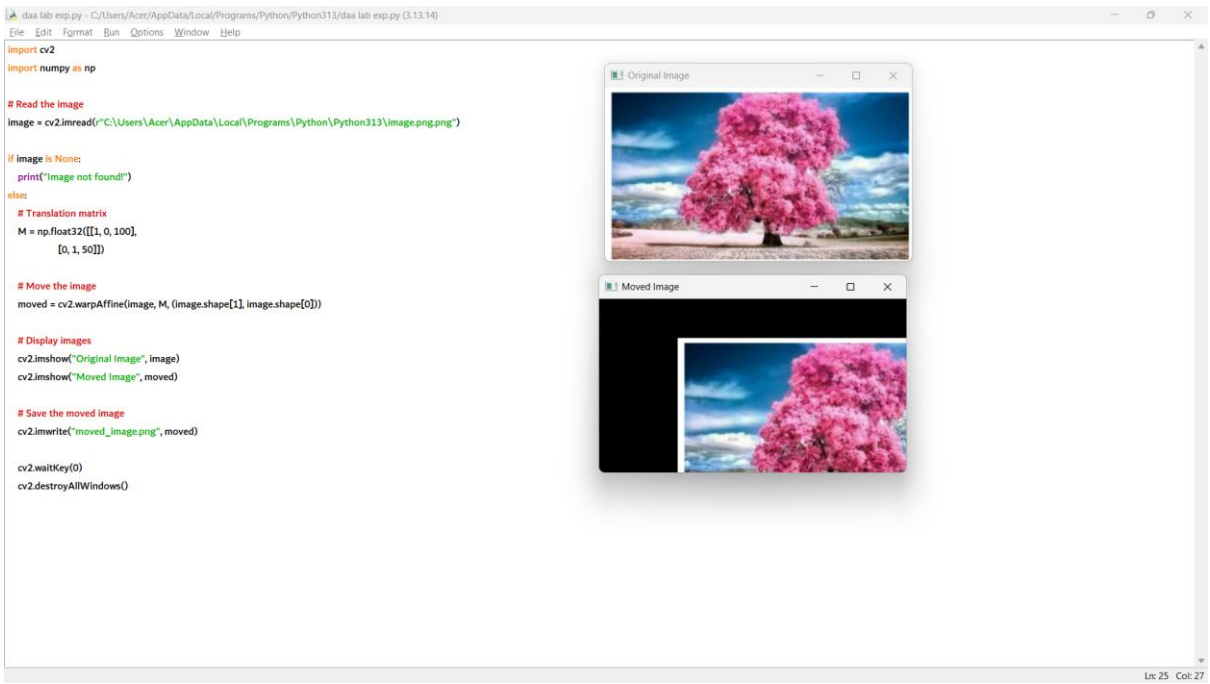
    M = np.float32([[1, 0, 100],

                    [0, 1, 50]])

    # Move the image

    moved = cv2.warpAffine(image, M, (image.shape[1], image.shape[0]))

```
# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Moved Image", moved)

# Save the moved image
cv2.imwrite("moved_image.png", moved)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



7. import cv2

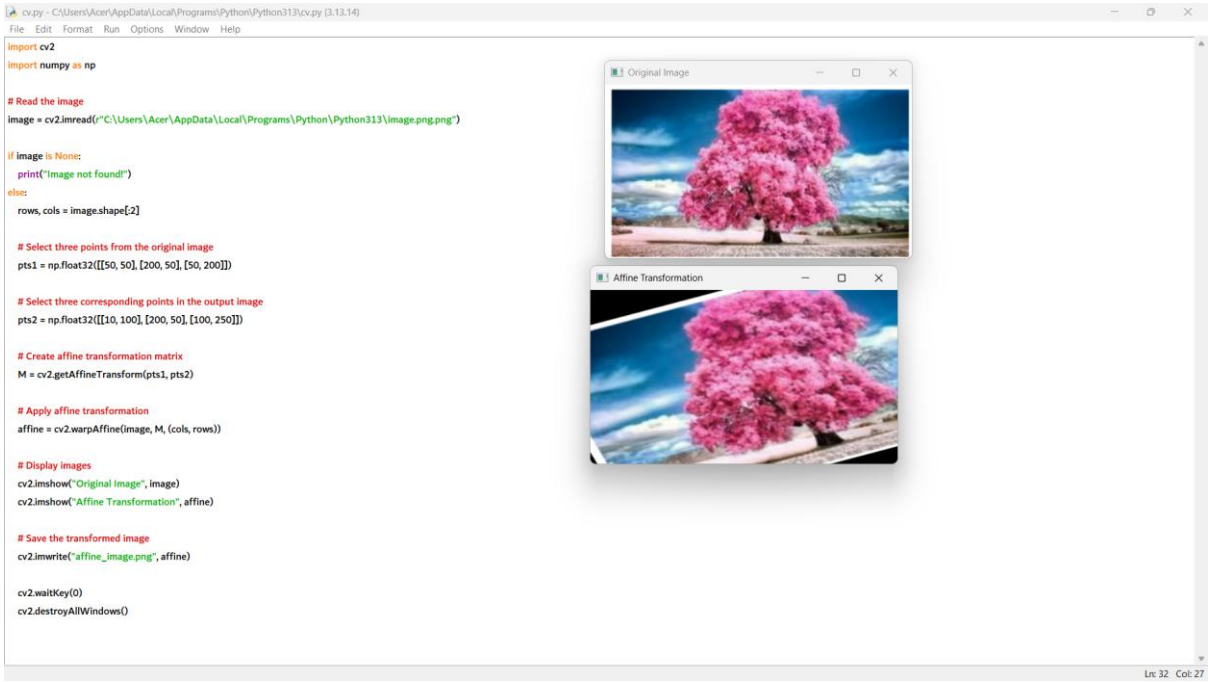
import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

```
    print("Image not found!")
else:
    rows, cols = image.shape[:2]

    # Select three points from the original image
    pts1 = np.float32([[50, 50], [200, 50], [50, 200]])

    # Select three corresponding points in the output image
    pts2 = np.float32([[10, 100], [200, 50], [100, 250]])

    # Create affine transformation matrix
    M = cv2.getAffineTransform(pts1, pts2)

    # Apply affine transformation
    affine = cv2.warpAffine(image, M, (cols, rows))

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Affine Transformation", affine)

    # Save the transformed image
    cv2.imwrite("affine_image.png", affine)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



8. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    rows, cols = image.shape[:2]

    # Four points in the original image

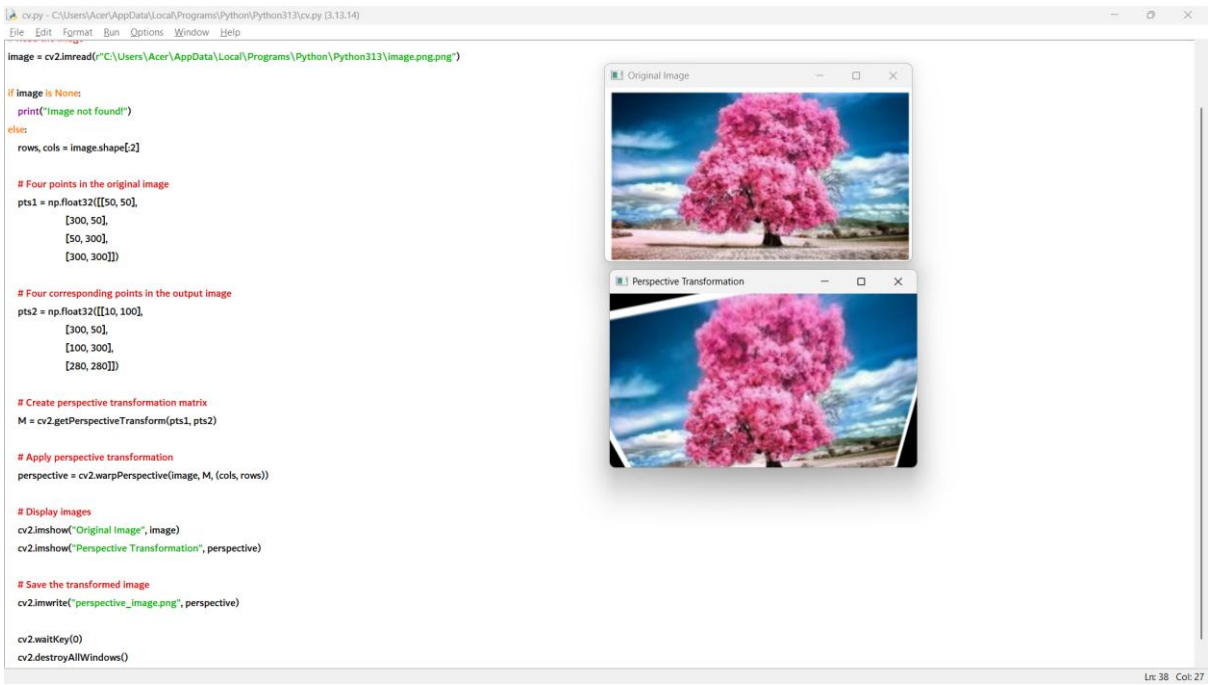
    pts1 = np.float32([[50, 50],

                       [300, 50],

                       [50, 300],

                       [300, 300]])

```
# Four corresponding points in the output image
pts2 = np.float32([[10, 100],
                   [300, 50],
                   [100, 300],
                   [280, 280]])

# Create perspective transformation matrix
M = cv2.getPerspectiveTransform(pts1, pts2)

# Apply perspective transformation
perspective = cv2.warpPerspective(image, M, (cols, rows))

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Perspective Transformation", perspective)

# Save the transformed image
cv2.imwrite("perspective_image.png", perspective)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



9. import cv2

import numpy as np

Read the video

video = cv2.VideoCapture(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\video.mp4")

if not video.isOpened():

    print("Video not found!")

else:

    while True:

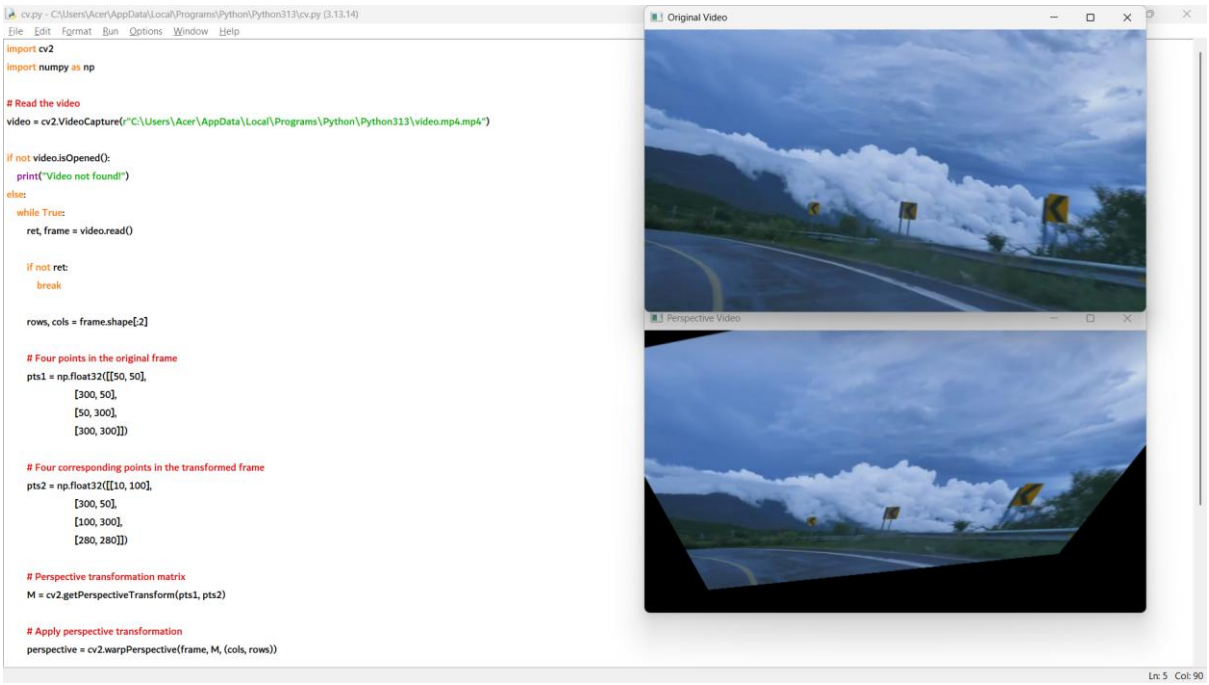
        ret, frame = video.read()

        if not ret:

            break

```
        rows, cols = frame.shape[:2]

        # Four points in the original frame
        pts1 = np.float32([[50, 50],
                           [300, 50],
                           [50, 300],
                           [300, 300]])

        # Four corresponding points in the transformed frame
        pts2 = np.float32([[10, 100],
                           [300, 50],
                           [100, 300],
                           [280, 280]])

        # Perspective transformation matrix
        M = cv2.getPerspectiveTransform(pts1, pts2)

        # Apply perspective transformation
        perspective = cv2.warpPerspective(frame, M, (cols, rows))

        # Display original and transformed video
        cv2.imshow("Original Video", frame)
        cv2.imshow("Perspective Video", perspective)

        # Press 'q' to quit
        if cv2.waitKey(30) & 0xFF == ord('q'):
            break

video.release()
```

cv2.destroyAllWindows()



10. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

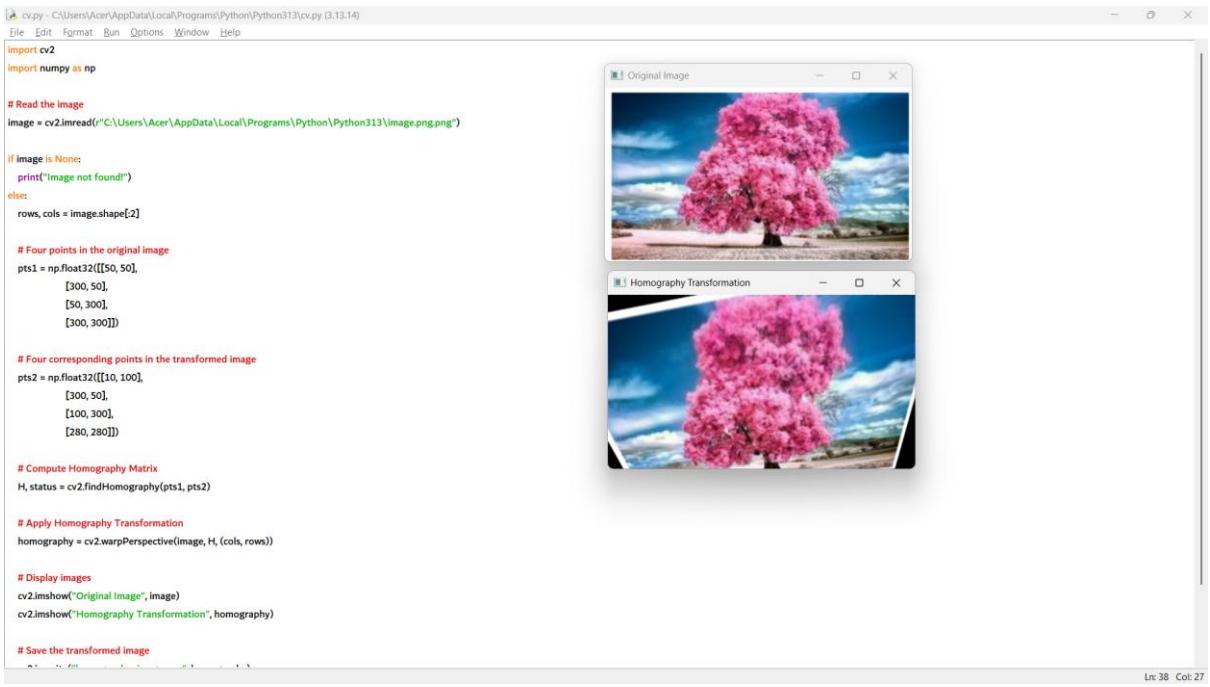
    rows, cols = image.shape[:2]

    # Four points in the original image

    pts1 = np.float32([[50, 50],

                       [300, 50],

```
                [50, 300],
                [300, 300]])

# Four corresponding points in the transformed image
pts2 = np.float32([[10, 100],
                [300, 50],
                [100, 300],
                [280, 280]])

# Compute Homography Matrix
H, status = cv2.findHomography(pts1, pts2)

# Apply Homography Transformation
homography = cv2.warpPerspective(image, H, (cols, rows))

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Homography Transformation", homography)

# Save the transformed image
cv2.imwrite("homography_image.png", homography)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



11. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    rows, cols = image.shape[:2]

    # Four points in the original image

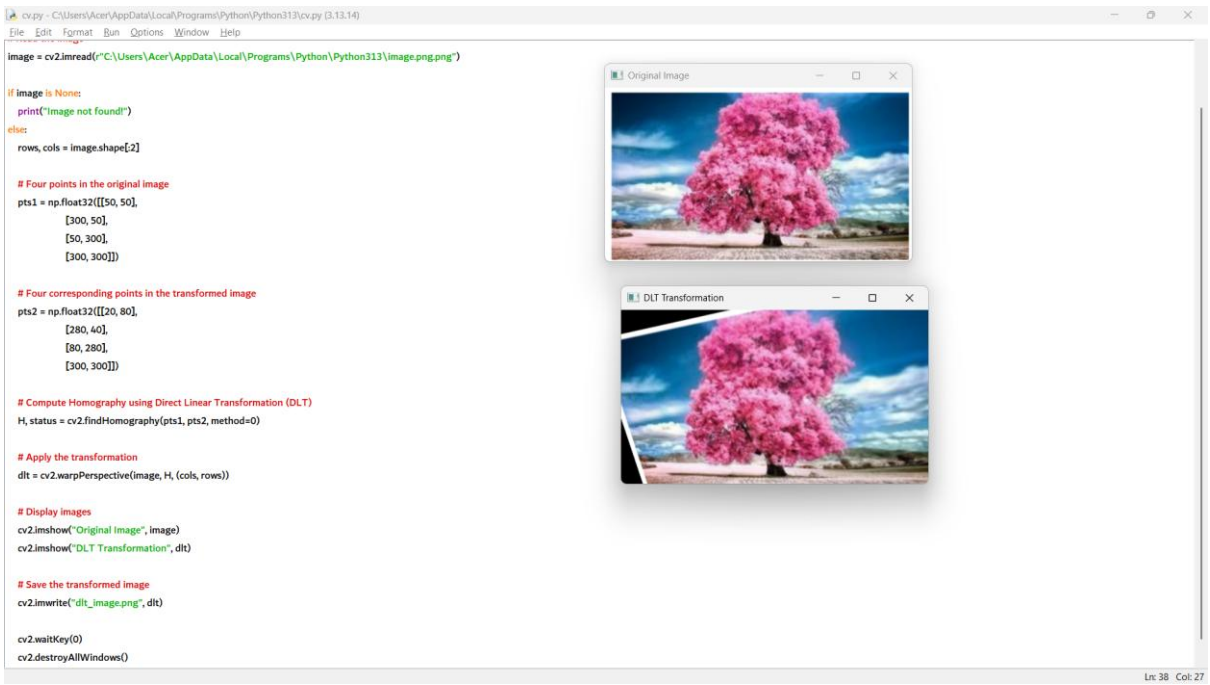
    pts1 = np.float32([[50, 50],

                       [300, 50],

                       [50, 300],

                       [300, 300]])

```
# Four corresponding points in the transformed image
pts2 = np.float32([[20, 80],
                   [280, 40],
                   [80, 280],
                   [300, 300]])

# Compute Homography using Direct Linear Transformation (DLT)
H, status = cv2.findHomography(pts1, pts2, method=0)

# Apply the transformation
dlt = cv2.warpPerspective(image, H, (cols, rows))

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("DLT Transformation", dlt)

# Save the transformed image
cv2.imwrite("dlt_image.png", dlt)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



12. import cv2

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Convert image to grayscale

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

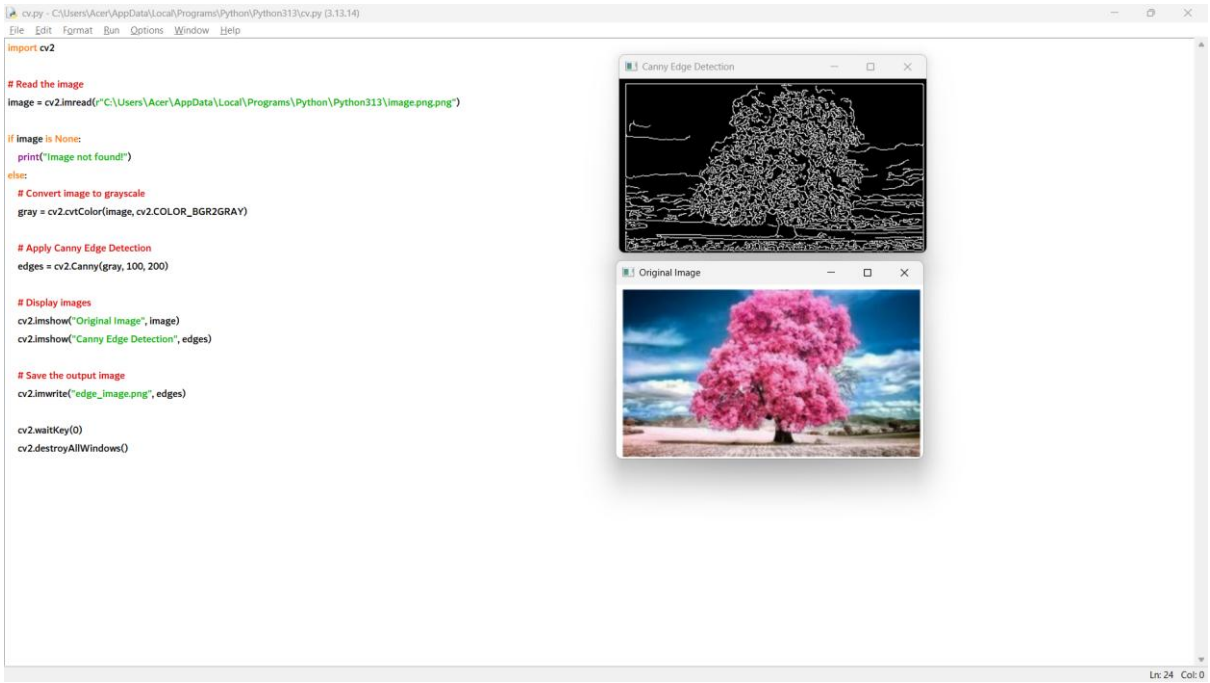
    # Apply Canny Edge Detection

    edges = cv2.Canny(gray, 100, 200)

    # Display images

    cv2.imshow("Original Image", image)

```
    cv2.imshow("Canny Edge Detection", edges)

    # Save the output image
    cv2.imwrite("edge_image.png", edges)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



13. import cv2

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

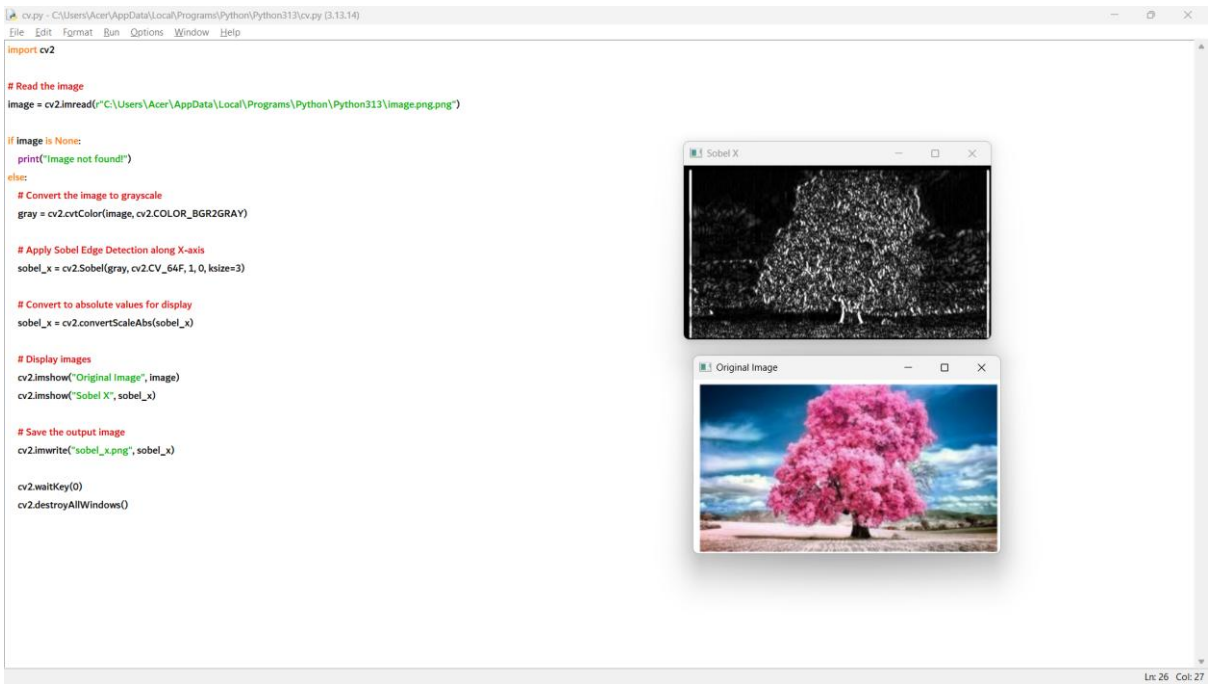
if image is None:

    print("Image not found!")

else:

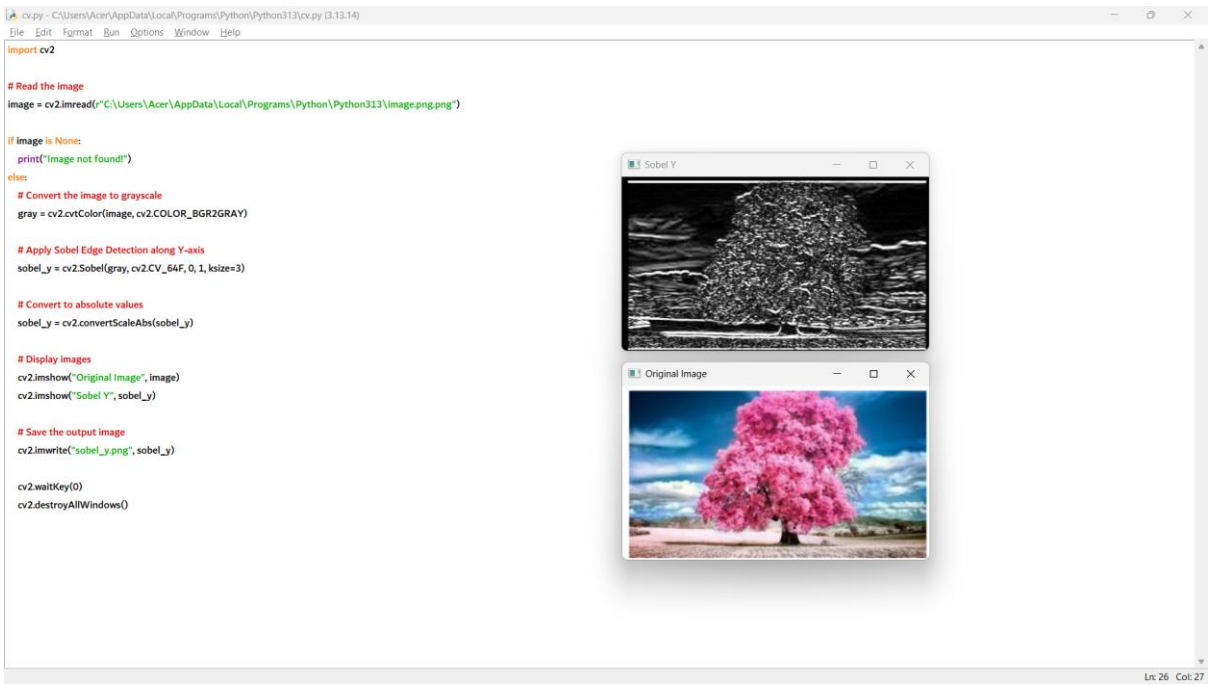
    # Convert the image to grayscale

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Sobel Edge Detection along X-axis
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

# Convert to absolute values for display
sobel_x = cv2.convertScaleAbs(sobel_x)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Sobel X", sobel_x)

# Save the output image
cv2.imwrite("sobel_x.png", sobel_x)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



14. import cv2

```
# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:
    print("Image not found!")
else:
    # Convert the image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Apply Sobel Edge Detection along Y-axis
    sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

    # Convert to absolute values
    sobel_y = cv2.convertScaleAbs(sobel_y)

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Sobel Y", sobel_y)

    # Save the output image
    cv2.imwrite("sobel_y.png", sobel_y)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



15. import cv2

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Convert the image to grayscale

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

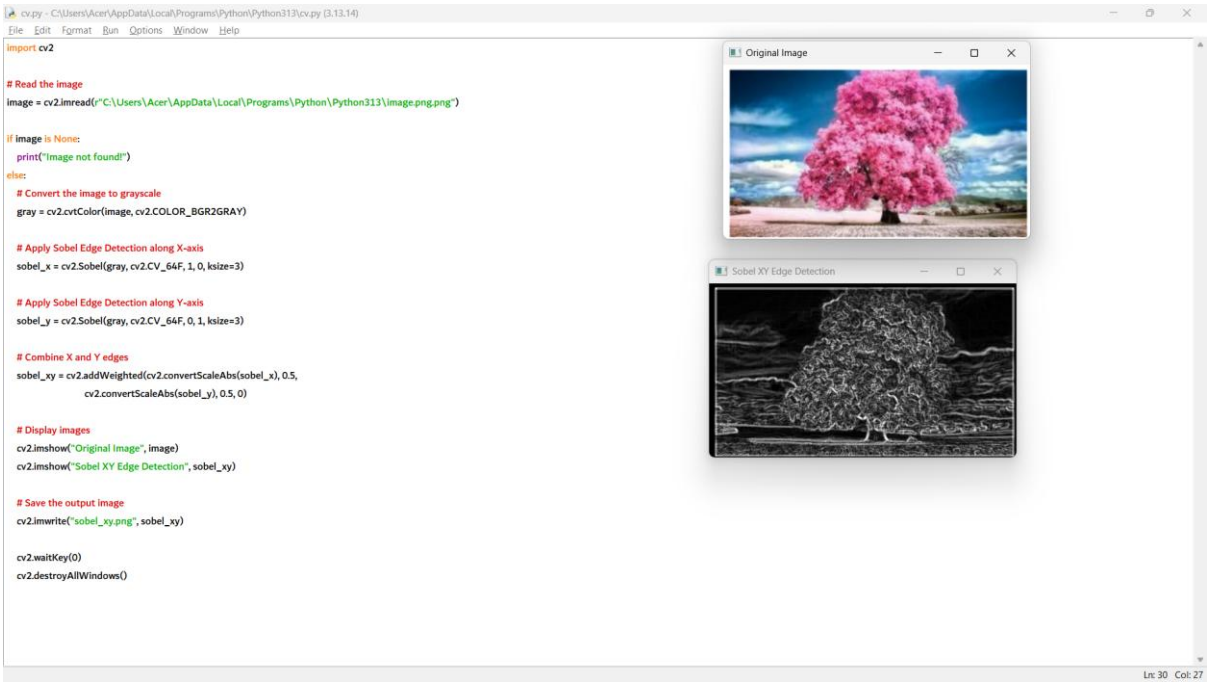
    # Apply Sobel Edge Detection along X-axis

    sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

    # Apply Sobel Edge Detection along Y-axis

    sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

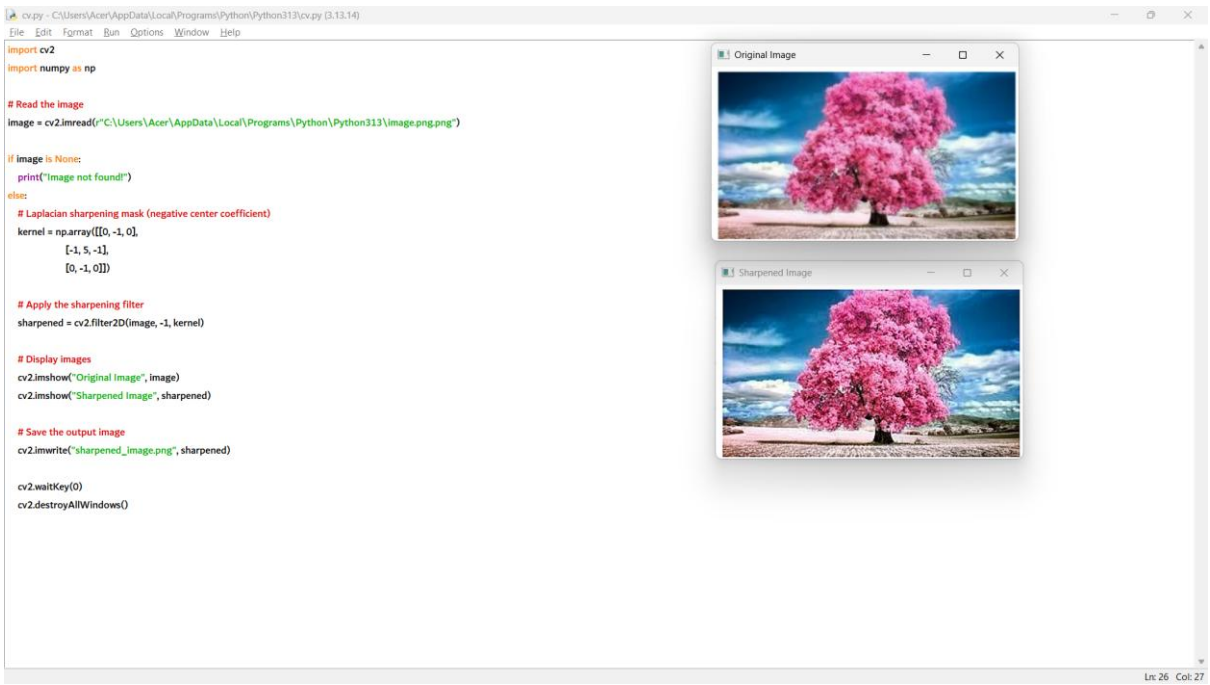
```
# Combine X and Y edges
sobel_xy = cv2.addWeighted(cv2.convertScaleAbs(sobel_x), 0.5,
                           cv2.convertScaleAbs(sobel_y), 0.5, 0)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Sobel XY Edge Detection", sobel_xy)

# Save the output image
cv2.imwrite("sobel_xy.png", sobel_xy)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



16. import cv2

import numpy as np

```
# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:
    print("Image not found!")
else:
    # Laplacian sharpening mask (negative center coefficient)
    kernel = np.array([[0, -1, 0],
                       [-1, 5, -1],
                       [0, -1, 0]])

    # Apply the sharpening filter
    sharpened = cv2.filter2D(image, -1, kernel)

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Sharpened Image", sharpened)

    # Save the output image
    cv2.imwrite("sharpened_image.png", sharpened)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



17. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Laplacian mask with diagonal neighbors

    kernel = np.array([[1, 1, 1],

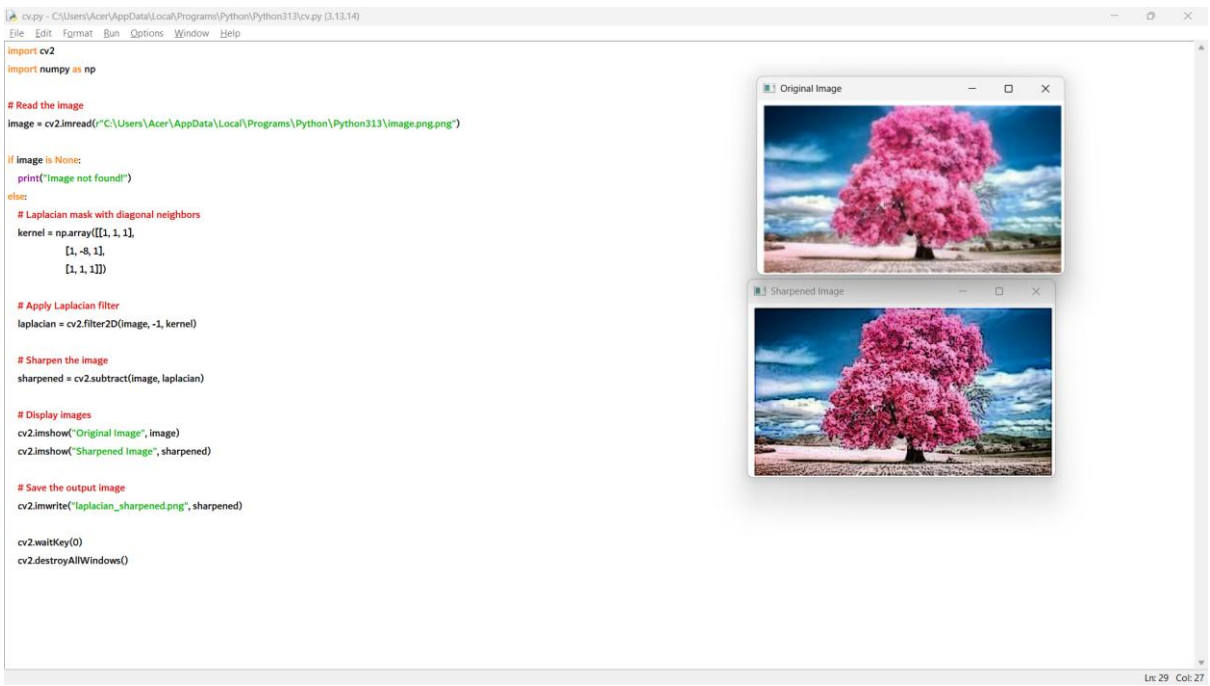
                       [1, -8, 1],

                       [1, 1, 1]])

    # Apply Laplacian filter

    laplacian = cv2.filter2D(image, -1, kernel)

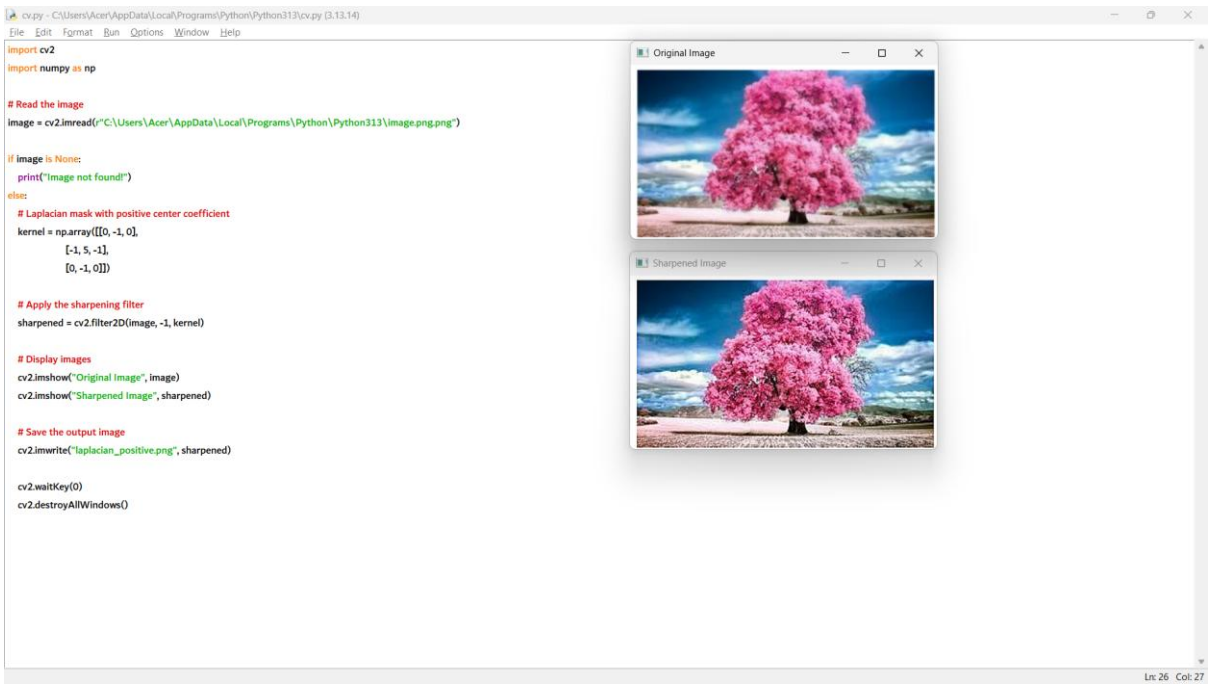
```
# Sharpen the image
sharpened = cv2.subtract(image, laplacian)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Sharpened Image", sharpened)

# Save the output image
cv2.imwrite("laplacian_sharpened.png", sharpened)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



18. import cv2

import numpy as np

```
# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:
    print("Image not found!")
else:
    # Laplacian mask with positive center coefficient
    kernel = np.array([[0, -1, 0],
                       [-1, 5, -1],
                       [0, -1, 0]])

    # Apply the sharpening filter
    sharpened = cv2.filter2D(image, -1, kernel)

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Sharpened Image", sharpened)

    # Save the output image
    cv2.imwrite("laplacian_positive.png", sharpened)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



19. import cv2

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Blur the image

    blurred = cv2.GaussianBlur(image, (5, 5), 0)

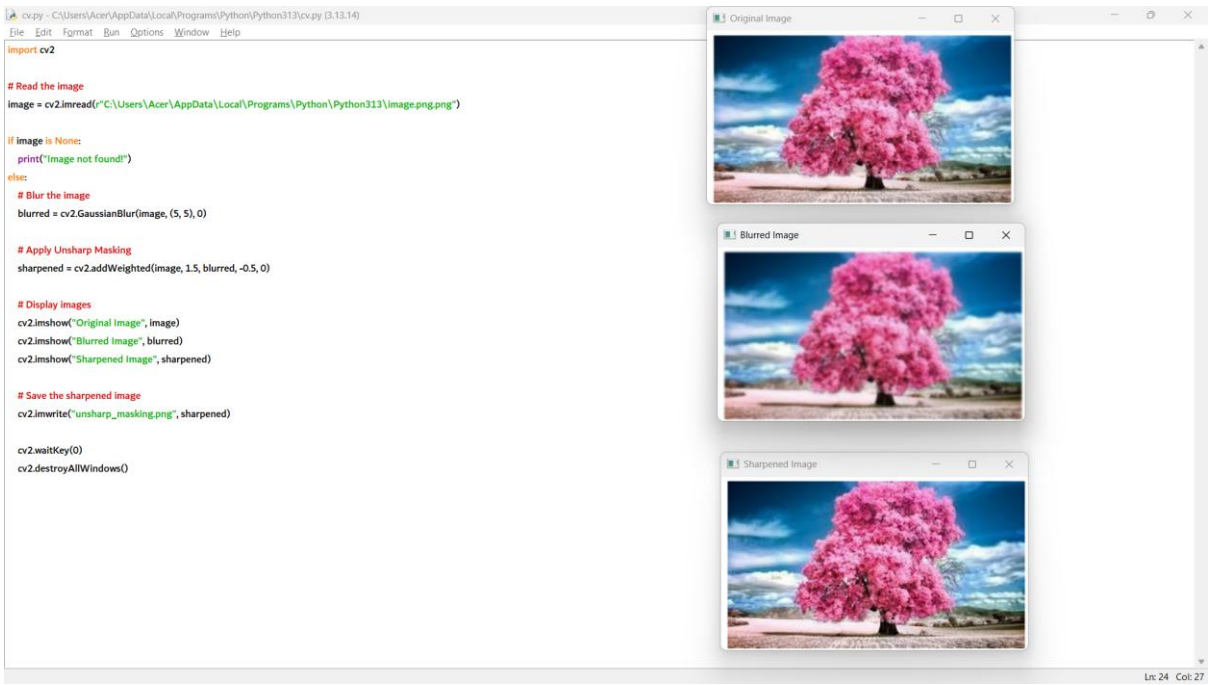
    # Apply Unsharp Masking

    sharpened = cv2.addWeighted(image, 1.5, blurred, -0.5, 0)

    # Display images

    cv2.imshow("Original Image", image)

```
cv2.imshow("Blurred Image", blurred)
cv2.imshow("Sharpened Image", sharpened)

# Save the sharpened image
cv2.imwrite("unsharp_masking.png", sharpened)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



20. import cv2

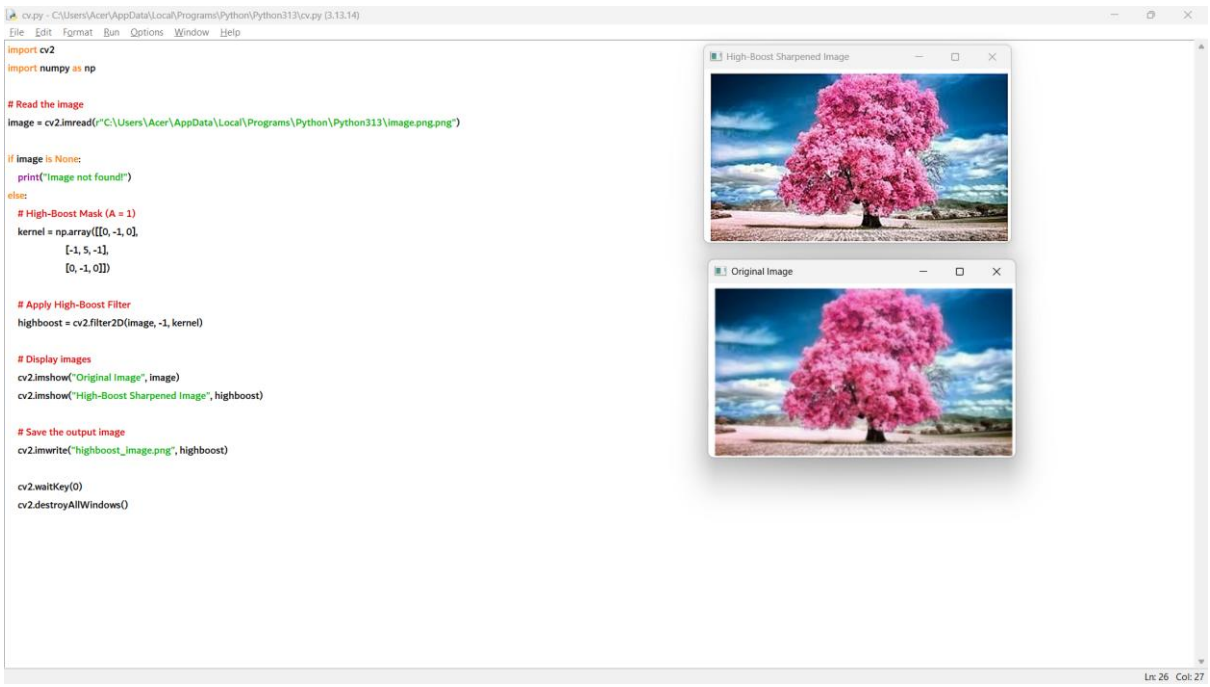
import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

```
    print("Image not found!")
else:
    # High-Boost Mask (A = 1)
    kernel = np.array([[0, -1, 0],
                       [-1, 5, -1],
                       [0, -1, 0]])

    # Apply High-Boost Filter
    highboost = cv2.filter2D(image, -1, kernel)

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("High-Boost Sharpened Image", highboost)

    # Save the output image
    cv2.imwrite("highboost_image.png", highboost)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



21. import cv2

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Convert image to grayscale

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

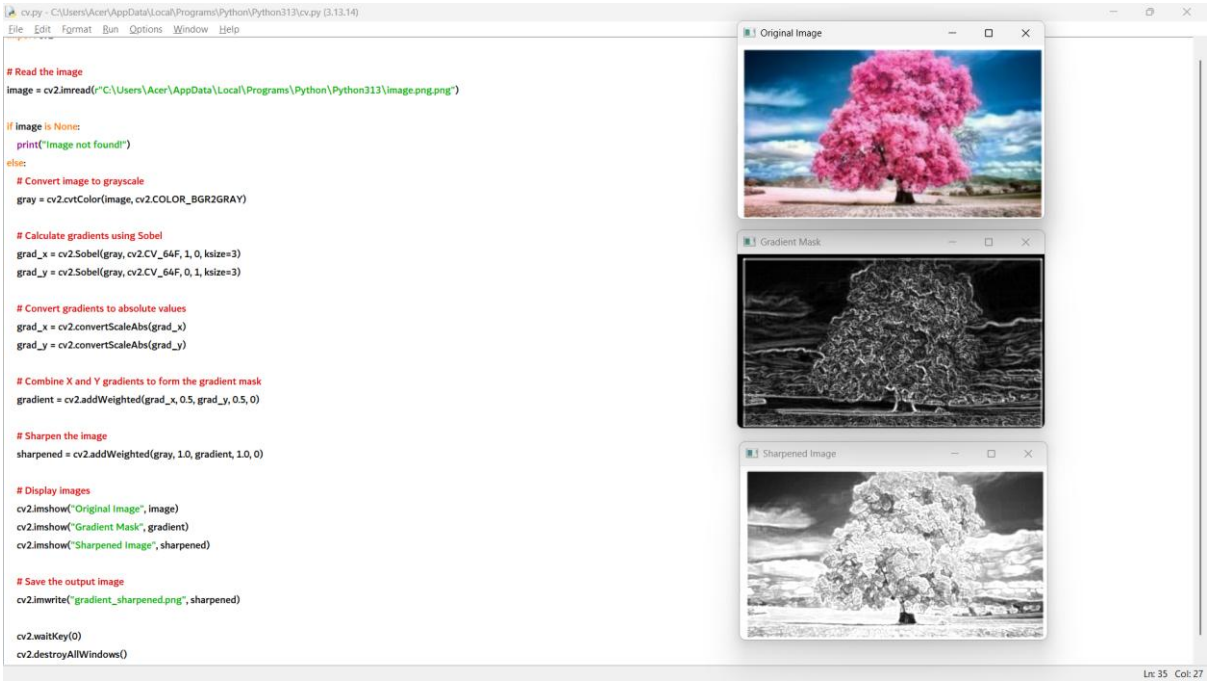
    # Calculate gradients using Sobel

    grad_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

    grad_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

    # Convert gradients to absolute values

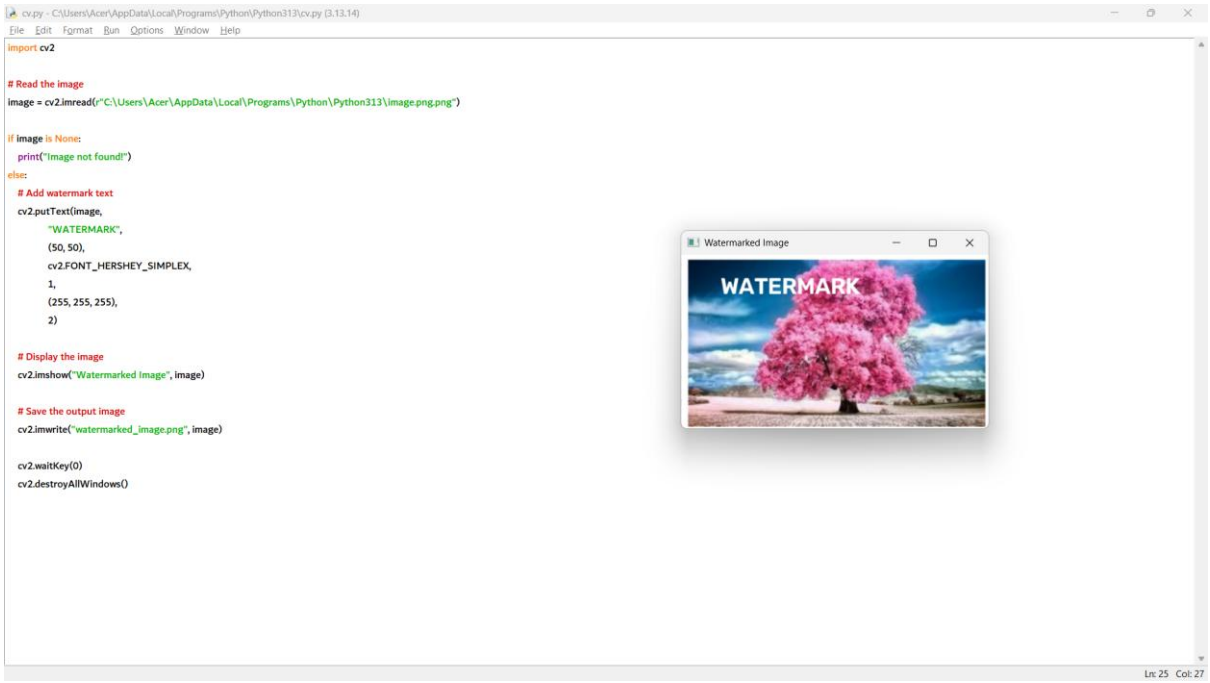
```
grad_x = cv2.convertScaleAbs(grad_x)
grad_y = cv2.convertScaleAbs(grad_y)

# Combine X and Y gradients to form the gradient mask
gradient = cv2.addWeighted(grad_x, 0.5, grad_y, 0.5, 0)

# Sharpen the image
sharpened = cv2.addWeighted(gray, 1.0, gradient, 1.0, 0)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Gradient Mask", gradient)
cv2.imshow("Sharpened Image", sharpened)

# Save the output image
cv2.imwrite("gradient_sharpened.png", sharpened)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



22. import cv2

```
# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:
    print("Image not found!")
else:
    # Add watermark text
    cv2.putText(image,
                "WATERMARK",
                (50, 50),
                cv2.FONT_HERSHEY_SIMPLEX,
                1,
                (255, 255, 255),
                2)
```

```
# Display the image
cv2.imshow("Watermarked Image", image)

# Save the output image
cv2.imwrite("watermarked_image.png", image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



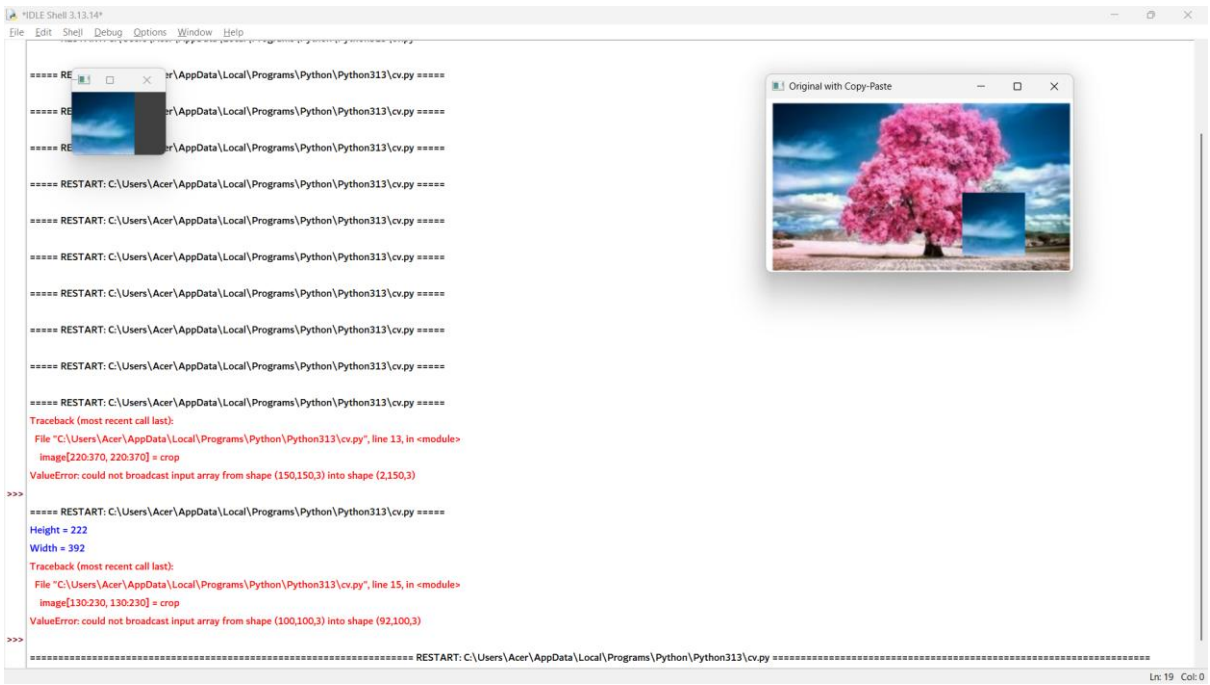
23. import cv2

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

```
    print("Image not found!")
else:
    # Crop a portion of the image
    crop = image[50:200, 50:200]

    # Copy and paste the cropped portion to another location
    image[220:370, 220:370] = crop

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Cropped Image", crop)

    # Save the output image
    cv2.imwrite("crop_copy_paste.png", image)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



24. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

    # Create a 5x5 kernel

    kernel = np.ones((5,5), np.uint8)

    # Apply erosion

    eroded = cv2.erode(image, kernel, iterations=1)

    # Display images

    cv2.imshow("Original Image", image)

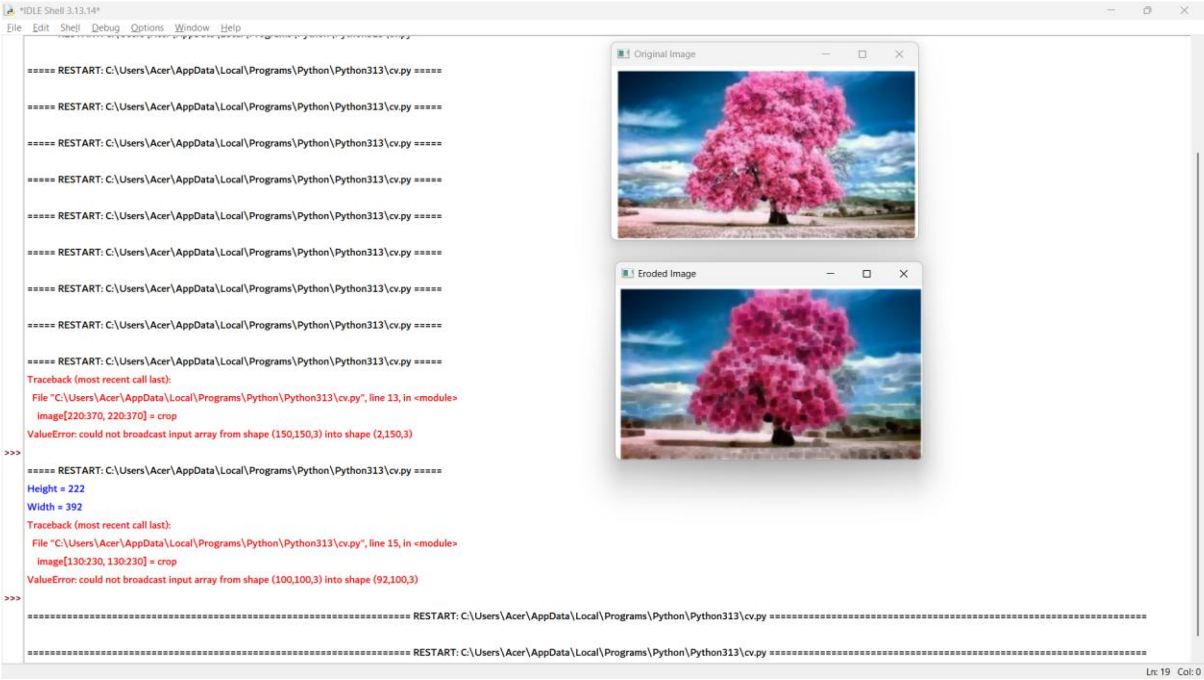
    cv2.imshow("Eroded Image", eroded)

    # Save the output image

    cv2.imwrite("eroded_image.png", eroded)

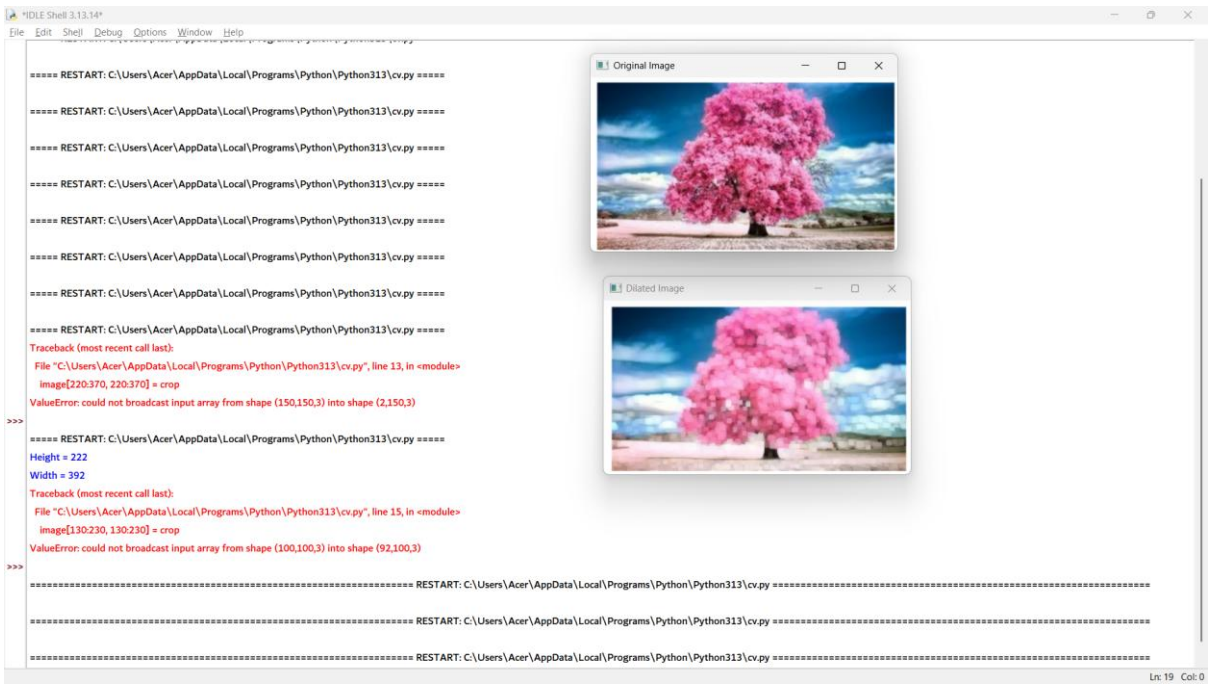
    cv2.waitKey(0)

    cv2.destroyAllWindows()



25.
```
import cv2
import numpy as np

# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:
    print("Image not found!")
else:
    # Create a 5x5 kernel
    kernel = np.ones((5,5), np.uint8)

    # Apply dilation
    dilated = cv2.dilate(image, kernel, iterations=1)

    # Display images
```

```
cv2.imshow("Original Image", image)
cv2.imshow("Dilated Image", dilated)

# Save the output image
cv2.imwrite("dilated_image.png", dilated)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



26. import cv2

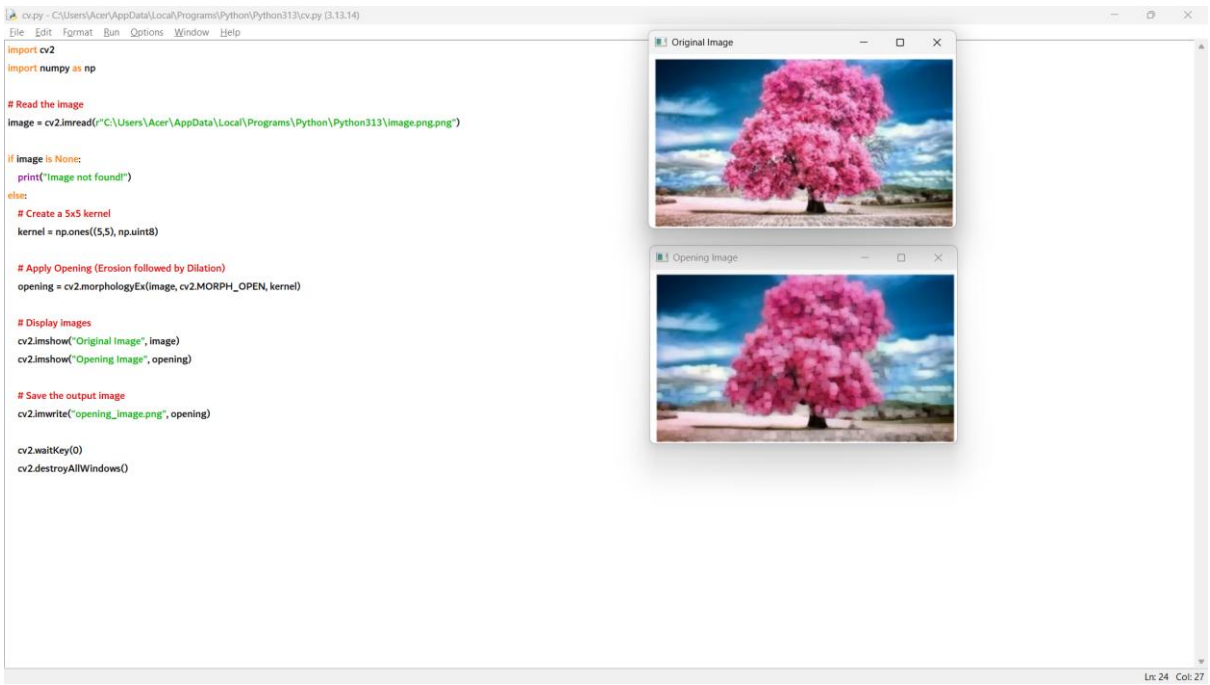
import numpy as np

Read the image

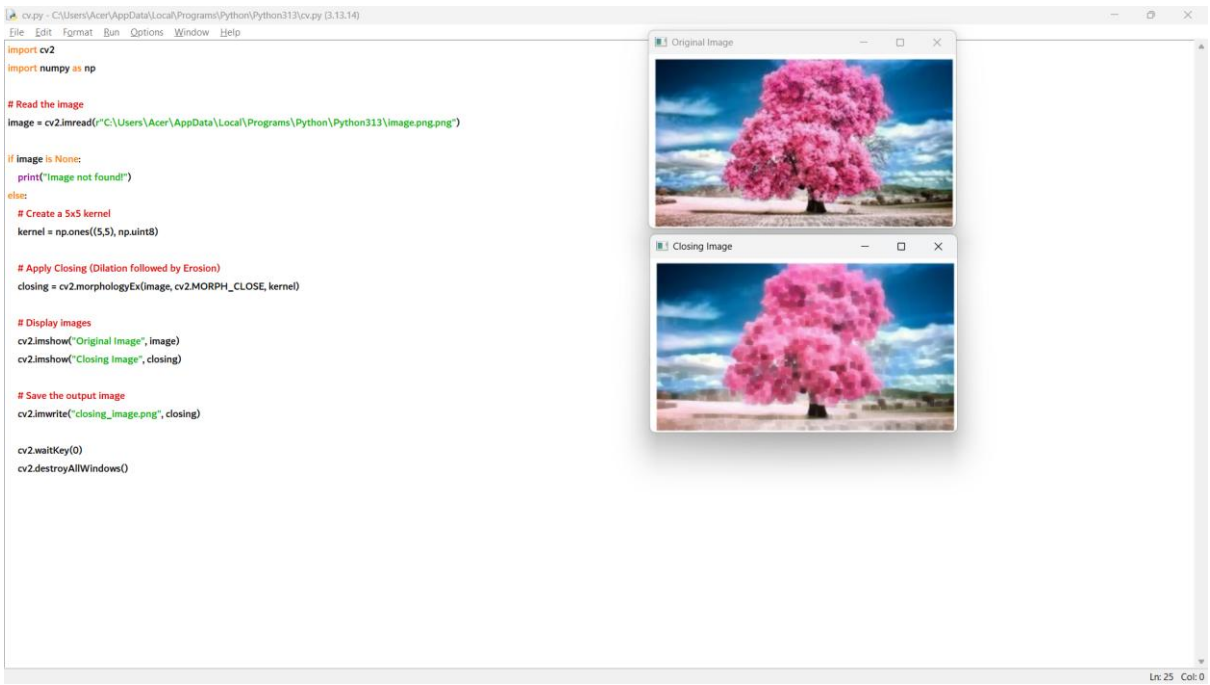
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

```
    print("Image not found!")
else:
    # Create a 5x5 kernel
    kernel = np.ones((5,5), np.uint8)

    # Apply Opening (Erosion followed by Dilation)
    opening = cv2.morphologyEx(image, cv2.MORPH_OPEN, kernel)

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Opening Image", opening)

    # Save the output image
    cv2.imwrite("opening_image.png", opening)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



27.
```
import cv2
import numpy as np

# Read the image in grayscale
img = cv2.imread('image.jpg', 0)

# Create a kernel
kernel = np.ones((5,5), np.uint8)

# Apply Closing operation
closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)

# Display images
cv2.imshow('Original Image', img)
cv2.imshow('Closing Image', closing)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



28. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

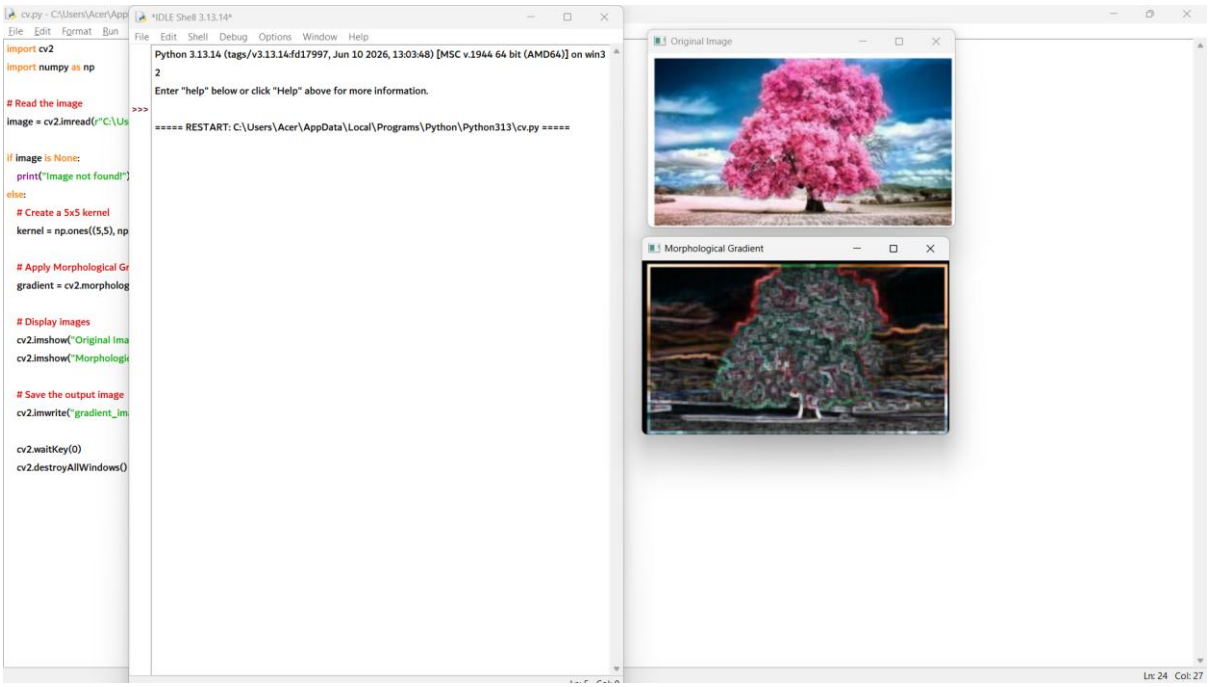
    # Create a 5x5 kernel

    kernel = np.ones((5,5), np.uint8)

    # Apply Morphological Gradient

    gradient = cv2.morphologyEx(image, cv2.MORPH_GRADIENT, kernel)

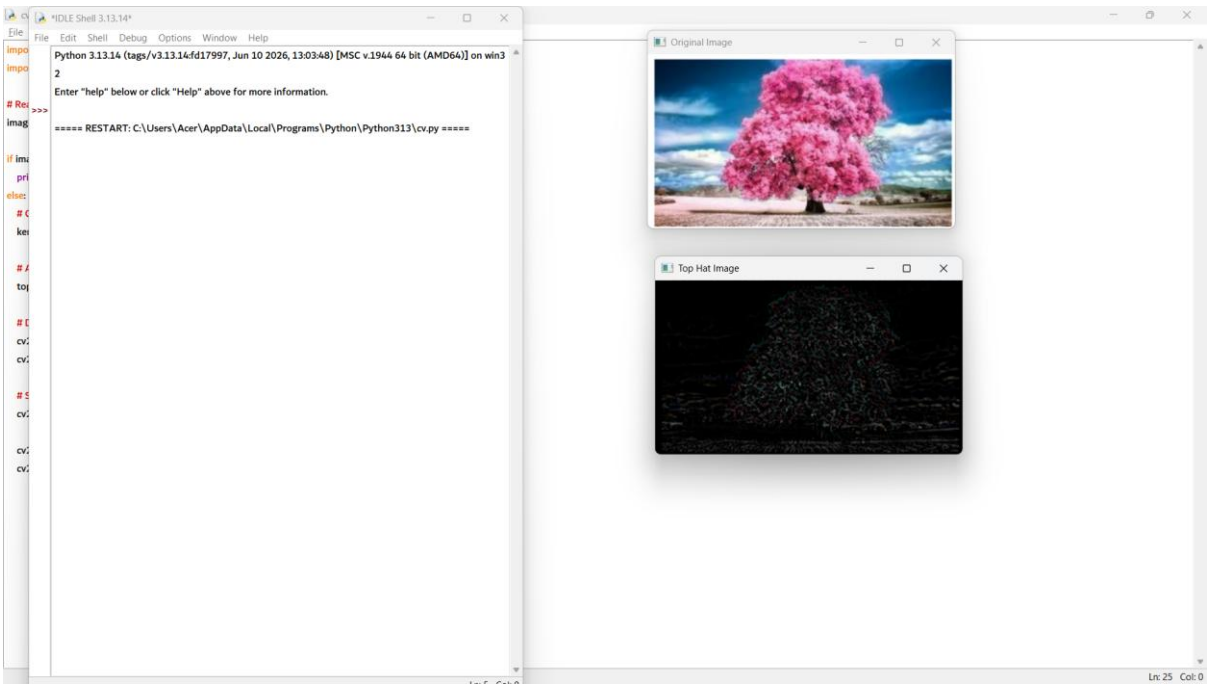
```
# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Morphological Gradient", gradient)

# Save the output image
cv2.imwrite("gradient_image.png", gradient)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



29. import cv2
import numpy as np

Read the image
image =

```
cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:
    print("Image not found!")
else:
    # Create a 5x5 kernel
    kernel = np.ones((5,5), np.uint8)

    # Apply Top Hat operation
    tophat = cv2.morphologyEx(image, cv2.MORPH_TOPHAT, kernel)

    # Display images
    cv2.imshow("Original Image", image)
    cv2.imshow("Top Hat Image", tophat)

    # Save the output image
    cv2.imwrite("tophat_image.png", tophat)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



30. import cv2

import numpy as np

Read the image

image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:

    print("Image not found!")

else:

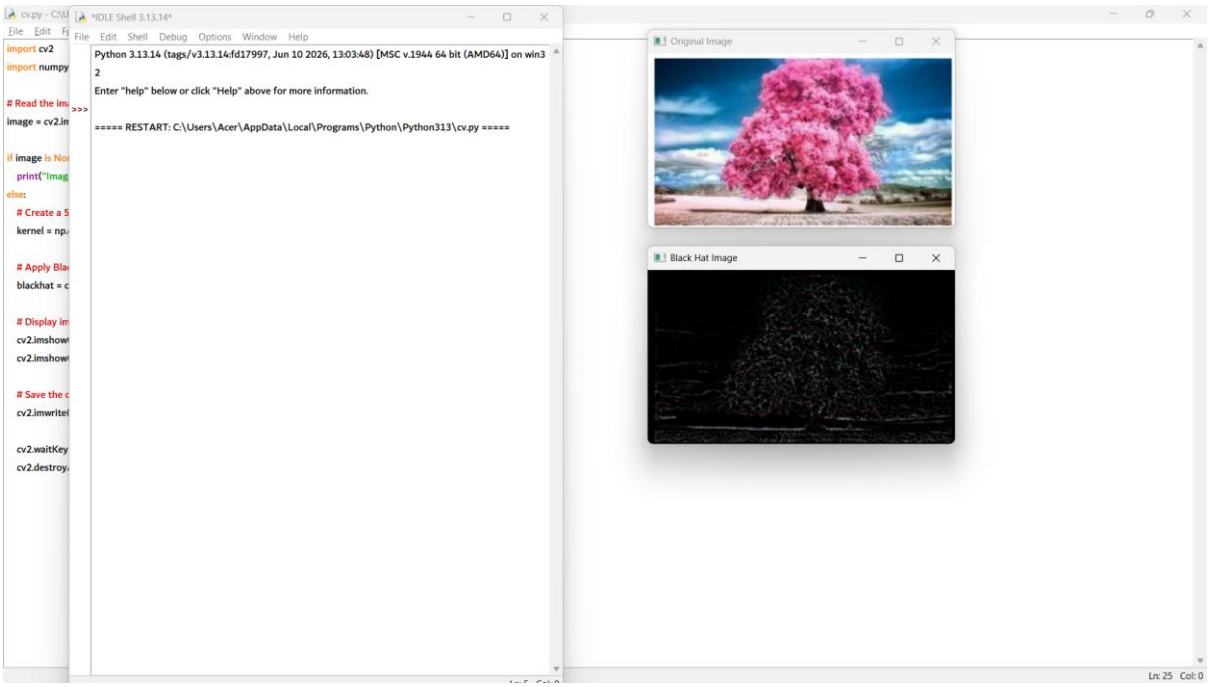
    # Create a 5x5 kernel

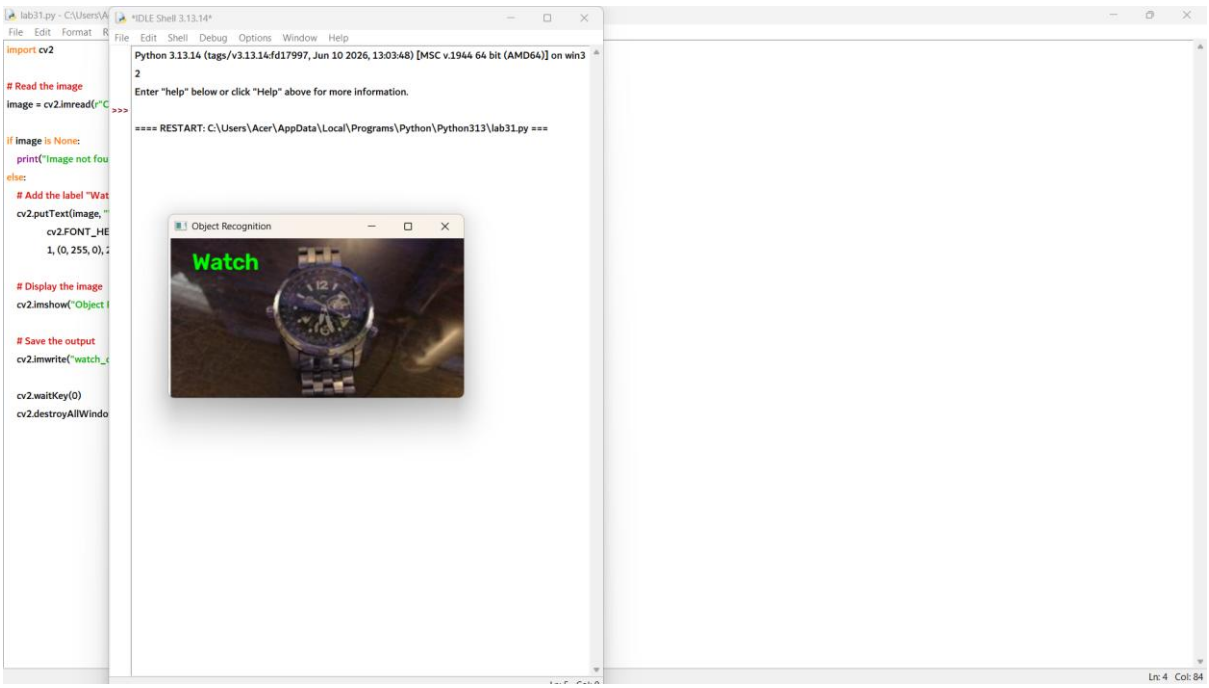
    kernel = np.ones((5,5), np.uint8)

    # Apply Black Hat operation

    blackhat = cv2.morphologyEx(image, cv2.MORPH_BLACKHAT, kernel)

```
# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Black Hat Image", blackhat)

# Save the output image
cv2.imwrite("blackhat_image.png", blackhat)

cv2.waitKey(0)
cv2.destroyAllWindows()
```





32. import cv2

Open the video

cap = cv2.VideoCapture(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\video.mp4")

if not cap.isOpened():

    print("Video not found!")

else:

    # Read all frames

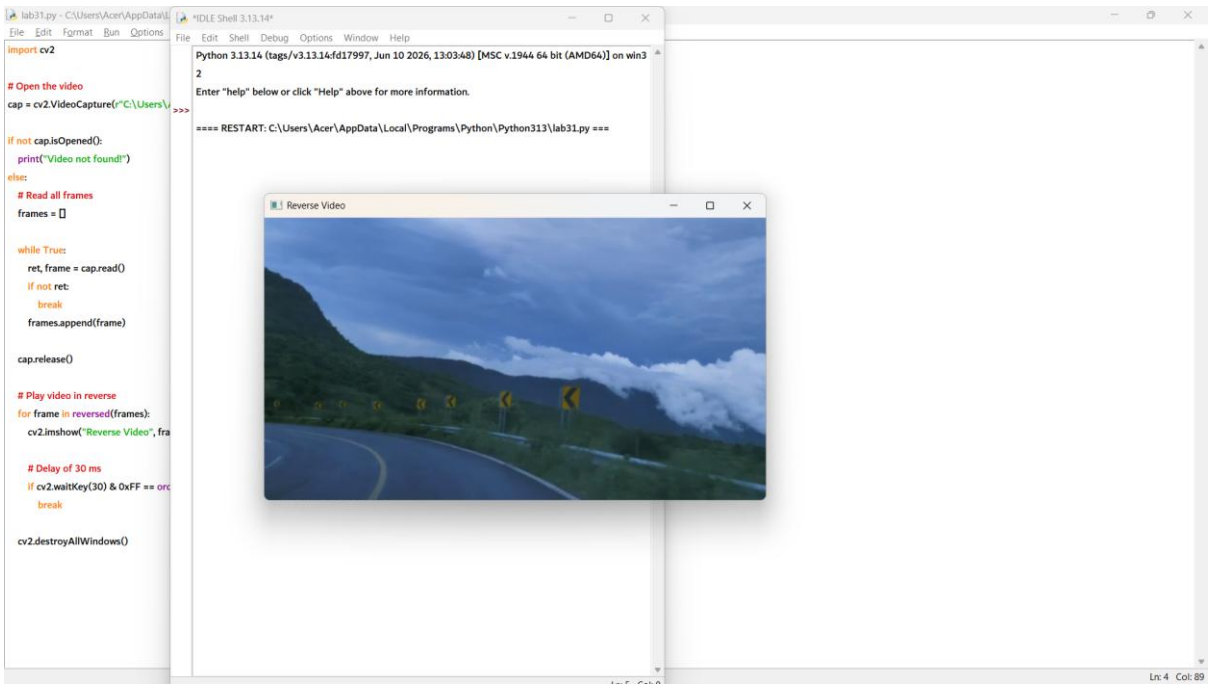
    frames = []

    while True:

        ret, frame = cap.read()

        if not ret:

```
        break
    frames.append(frame)

cap.release()

# Play video in reverse
for frame in reversed(frames):
    cv2.imshow("Reverse Video", frame)

    # Delay of 30 ms
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cv2.destroyAllWindows()
```



33. import cv2

```
# Load Haar Cascade classifier for face detection
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\face.jpg")

if image is None:
    print("Image not found!")
else:
    # Convert image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

    # Draw rectangles around faces
    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)

    # Display the output
    cv2.imshow("Face Detection", image)

    # Save the output image
    cv2.imwrite("face_detected.png", image)
```

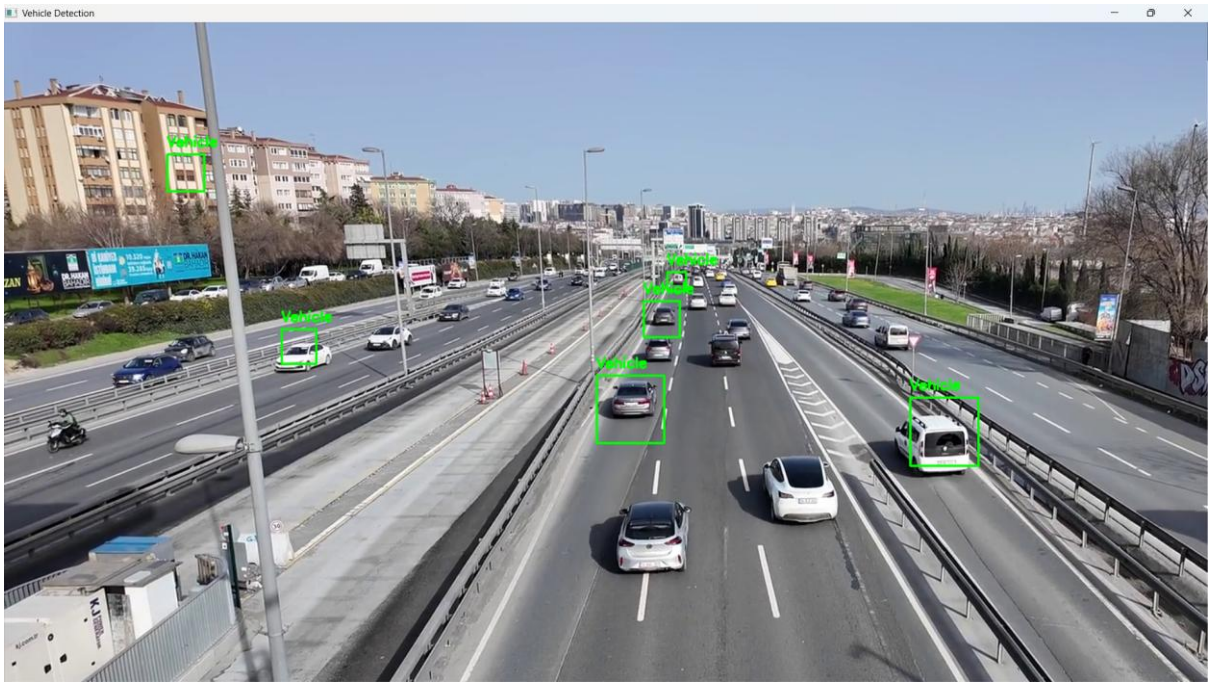
```
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

34. import cv2

```
# Load the vehicle cascade classifier
car_cascade = cv2.CascadeClassifier(
    r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\cars.xml"
)

# Open the video
cap = cv2.VideoCapture(
    r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\video.mp4"
)
```

```
if not cap.isOpened():
    print("Video not found!")

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # Convert frame to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect vehicles
    cars = car_cascade.detectMultiScale(gray, 1.1, 2)

    # Draw rectangles around vehicles
    for (x, y, w, h) in cars:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(frame, "Vehicle", (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.6, (0, 255, 0), 2)

    # Display the video
    cv2.imshow("Vehicle Detection", frame)

    # Press 'q' to quit
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break
```

```
cap.release()
cv2.destroyAllWindows()
```



35. import cv2

```
# Read the image
image = cv2.imread(r"C:\Users\Acer\AppData\Local\Programs\Python\Python313\image.png.png")

if image is None:
    print("Image not found!")
else:
    # Draw a rectangle
    cv2.rectangle(image, (50, 50), (250, 250), (0, 255, 0), 2)
```

```
# Extract the object inside the rectangle
object_img = image[50:250, 50:250]

# Display images
cv2.imshow("Image with Rectangle", image)
cv2.imshow("Extracted Object", object_img)

# Save the extracted object
cv2.imwrite("extracted_object.png", object_img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```